2027年热门会议推荐

具体咨询：白老师 13516196409

智驱产品技术领航

AI 时代产品和软件开发前沿系列峰会

核心定位

锚定AI 技术革新前沿，聚焦大模型与智能体工程化落地，解构 AI 时代软件工程架构革新路径，打通 AI 产品从技术突破到产品商业化验证的全链路。

关键词

- 大模型驱动
- 技术趋势
- Agent工程化
- 架构革新
- AI产品0-1
- 商业化验证

PM×AI 产品力领航者大会

2027年4月23-24·杭州

详情查看官网
演讲申报｜参会购票

AgenticAICon 智能体应用与架构工程大会

2027年6月18-19·上海

详情查看官网
演讲申报｜参会购票

AgenticAICon 智能体应用与架构工程大会

2027年11月5-6日·深圳

4月 · 5月 · 6月 · 9月 · 10月 · 11月

AI 赋能提质增效

QECon全球软件质量&效能大会

QECon 全球软件质量&效能大会

2027年5月21-22·深圳

QECon 全球软件质量&效能大会

2027年9月10-11日·上海

QECon 全球软件质量&效能大会

2027年10月15-16日·北京

演讲申报｜参会购票
详情查看官网

核心定位

聚焦 AI 技术在软件研发全流程的落地应用，以“质量保障&效能提升”为双驱动，助力研发团队实现软件研发的智能化转型。

关键词

- AI 赋能研发
- 效能提升
- 测试驱动
- 平台工具链
- 质量保障
- .....

生产级Agent设计蓝皮书·行业案例卷

"蓝皮书共建计划"启动——行业与场景专家 招募中

白皮书定义模式，蓝皮书记录落地。
把你的生产案例，写进行业参照系。
你的案例，很可能是这个行业第一份模式应用实践指南。

交流地点：《艾普·AI生产力研究院》展位咨询

扫码参与

共建计划

以真实案例为基础，持续组织行业与场景的 Agent 设计模式应用撰写。蓝皮书不写概念，只记录具名系统——它面对什么约束、踩过什么失败、为什么这样选架构、哪些经验可以迁移到你的行业。

我们招募这样的行业与场景专家

主导或深度参与过 Agent 系统建设，有真实跑在生产环境的案例；
在强监管、高可靠或大规模部署约束下做过工程取舍（金融、制造、政务、医疗、能源、电商等皆宜）；
愿意把一段经验，沉淀成可评审、可迁移的模式案例。

参与共建，你将获得

署名蓝皮书案例贡献者，成果进入 ADPS 公共技术资料长期存档；
用白皮书的模式语言系统梳理自己的实践，让团队经验成为行业参照；
参与共同体模块研讨与模式评审，共同界定下一版模式的边界；
与已在共同体的腾讯、蚂蚁、百度、京东等一线专家跨行业互审。

《企业级 Agent 设计白皮书》发布

纸质版 & 电子版 同步上线

领取地点:《艾普·AI生产力研究院》展位

Agent 正在从「会回答问题的模型」变成「能参与业务流程的系统」。模型仍是引擎，但决定它在生产中稳不稳的，是周围这套工程结构。本白皮书给出智能体工程的可复用架构词汇与评审标准。

27	7×6	3	5	6
矩阵模式规范	认知功能 × 执行拓扑	横切工程面 X1–X3	工程契约	步选型法

智能体工程的「GoF设计模式」

27个可复用Agent设计模式双轴框架

企业级智能体架构评审与选型实战手册

双轴框架

纵轴 · 认知功能

Agent 在做什么类型的工作

感知 Perception　记忆 Memory　推理 Reasoning　行动 Action

反思 Reflection　协作 Collaboration　治理 Governance

横轴 · 执行拓扑

数据与控制怎么流动

链式 · 顺序　路由 · 分流　并行 · 并发　编排 · 协调

循环 · 迭代　层级 · 分层

获取白皮书电子版

艾普·AI生产力研究院
Artificial Intelligence Institute of Productivity

Artificial Intelligence Institute of Productivity · 中立 · 开放 · 共创

面向 AI 原生软件工程的开放研究与实践组织

驾驭智能·定义 AI 时代新质生产力

深度承接《"人工智能+软件"专项行动实施方案》的专业化研究与交流平台

软件质量与效能 **智能体工程与应用** **六大研究方向** **五大核心模块**

艾普 · AI 生产力研究院（AIIP）是一个中立、开放、共创的专业化研究与交流平台。研究院依托在软件质量与效能、智能体工程与应用等前沿领域的长期实践积累与前瞻研究底蕴，汇聚资深技术专家与企业研发团队等各方力量，共同推进行业能力体系建设、参考标准研究与评估框架研制，致力于成为支撑智能软件产业新质生产力发展的专业化交流平台。

更多详情请查看官网：https://aiip.org.cn/

关注官网

申请加入

与先行者同行，共赴 AI 生产力浪潮

六大研究方向

AI for SE
智能软件工程

AI赋能软件开发全流程、AI Coding 生产力与质量保障、智能测试工程，对标智能编程工具链与智能化技改。

度量体系
质效度量与治理

AI 质效度量模型、Token 成本治理、ROI 分析框架，以及成熟度评估参考框架的研究与共建。

ADPS
Agent 设计模式

Agentic 设计模式、多智能体架构、AI 系统技术栈与最佳实践沉淀，对标技能包与应用商店。

FDE
前线部署工程师

工程实践方法论、行业场景适配、企业落地案例与效能提升路径，对标就业友好与重点行业落地。

SE for AI
AI 工程化落地

AI 系统全生命周期工程化方法，可靠性与质量保障，以及 MLOps / LLMOps 交付与治理框架。

AI for X
AI 生产力

AI深度赋能千行百业，从软件研发走向办公、运营、数据、科研等真实业务价值场景，把工程能力转化为可度量的生产力。

AI 质效宣言

发起单位：QECon组委会 QECon Organizing Committee

从共识到行动，让宣言融入每一次研发实践

回归研发本质 **重塑人机协作** **推动行业共识**

协作共生 高于 局部取代
长效质量 高于 短期提效
适应进化 高于 固化流程
价值创造 高于 技术崇拜
数据驱动 高于 经验依赖
开放共享 高于 封闭自建

加入共建计划
获取电子画面

两层能力递进 · 一层工程底座

AI 增强软件研发全景架构

AI 嵌入 SDLC · 效能提升

• 人主导 • AI 协同

需求
- 行业调研与趋势分析
- 创意启发与功能探索
- 需求分析与规格细化
- 需求检查与内容评审

设计
- 代码理解与逻辑探测
- 技术选型与架构设计
- 接口设计与数据模型生成
- 技术方案评审与风险分析

开发
- 代码生成与缺陷解决
- 氛围编程与规约编程
- 单元测试生成与执行
- 代码评审与问题修复

测试
- 测试用例与数据生成
- 测试脚本修复与自愈
- 端到端测试生成与维护
- 缺陷发现与根因分析

CI / CD
- 构建与部署脚本生成
- 流水线失败诊断与修复
- 无人值守部署与发版
- 依赖升级与漏洞修复

运维 / 运营
- 告警聚合与摘要生成
- 故障分析与定位
- 故障处理与系统自愈
- 用户体验与反馈洞察

AI 研发范式变革 · 效能跃迁

• AI 主导 • 人监督

工程实践
- 驾驭工程 （模型决定上限，Harness 提升底线）
- 知识工程 （跨仓库代码与文档索引 · 沉淀复用）
- 验证闭环 （生成 → 验证 → 修正）
- 质量门禁 （构建多层次质量防护网）

流程重构
- 意图驱动、面向结果的交付 （替代多阶段接力）
- 对齐在前、执行在后 （人谈清边界，AI 自主实现）
- 需求即规格、证据即信任 （规格成为人机契约）
- 快速原型、持续迭代 （沿用敏捷 · 精益研发思想）

组织重塑
- 交付层抹平前后端分工 （跨栈交付 · 功能负责人）
- 架构师专注标准与门禁 （定标准 · 守门禁）
- 设计师与产品自主交付 （部分需求不经研发）
- 先锋队试点后规模复制 （先跑通再逐步推广）

效能度量
- 结果面：多 · 快 · 好 · 省
- 过程面：渗透率 · 成熟度 · 使用效果
- 数据源：代码 · 会话 · 配置 · 需求 · 监控
- 北极星：需求交付周期 · 单位 token 有效价值

AI 工程底座 · SE4AI （Software Engineering for AI）

• 自下而上 • 决定能力上限

模型层
- 多模型接入与路由 （模型网关 · 按能力与成本择优）
- 能力分级与降级 （强弱模型分级 · 故障回退）
- 领域微调与私有化 （微调训练 · 内网部署）
- 模型评测与选型 （基准跑分 · 场景实测对比）

Agent 层
- Agent 架构与编排 （单/多 Agent · 规划与控制流）
- 上下文工程与记忆 （上下文组装压缩 · 长短记忆）
- 运行时沙盒与工具接入 （隔离执行 · MCP 与技能调用）
- 灰度发布与回滚 （能力评估达标才放量 · 版本化 · 自动回退）

生态层
- 技能与智能体市场 （技能复用 · Agent 共享流通）
- MCP Hub 与工具生态 （工具目录 · 权限与调用治理）
- 知识资产与语料沉淀 （业务知识 · 代码知识 · 过程知识）
- 检索服务与知识图谱 （语义检索 · 图谱关系推理）

贯穿三层 | **安全与治理** 数据脱敏 · 提示注入防护 · 最小授权 · 输出合规 | **可观测性** 决策链路追踪 · 上下文溯源 · 失败归因 | **能力评估** 任务成功率 · 人工接管率 · 单任务成本

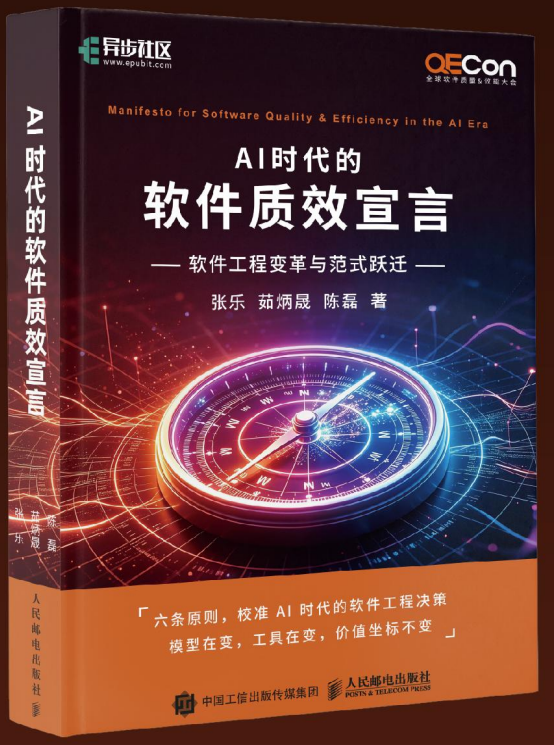
为每一位实践者提供一组稳定的价值坐标——当工具在变、模型在变、最佳实践在变的时候，这些原则帮你判断什么是不变的、什么是值得坚守的。

AI 时代的软件质效宣言

新书发布

— 软件工程变革与范式跃进 —

扫码订阅



系列电子书结构·三篇十九章 (道 · 术 · 势 —— 一条完整的 AI 研发变革路径)

道

第一篇 · 解读篇 (1-8 章)

确立价值观与原则：理解 AI 时代软件质效的本质，重建研发协作的信念底座，是全书的「道」。

术

即将发布

第二篇 · 单点提效篇 (9-14 章)

覆盖需求分析 → 架构设计 → 编码实现 → 测试验证 → CI/CD → 运维运营的关键环节，打通 AI 赋能研发全链路，是全书的「术」。

势

即将发布

第三篇 · 范式篇 (15-19 章)

当 AI 贯穿整条研发链路之后，组织形态本身将如何变革？从团队结构到协作范式，预见未来，是全书的「势」。

—— 主编 ——

张乐
腾讯
智能化软件工程
资深技术专家

茹炳晟
腾讯研究院
高级技术顾问

陈磊
前京东测试架构师

—— 作者 ——

王玉霞
北京兴云数科
过程改进专家

李佳奇
去哪儿旅行
技术总监

牛万鹏
百度
Comate研发经理

张晔
腾讯端服务质效
技术负责人

张涛
快手质量通道委员

付求爱
华为云
AIOps技术负责人

李萌
珊瑚智成
创始人&CEO

张刚
上海英慕科技
资深技术专家

张兆鑫
OPPO AI架构师

王磊
华为 AI技术专家

主办方：智盟创课

智盟创课（北京）科技有限公司（简称“智盟 Stu.”），作为专业IT产研培训服务供应商与AI时代数字化人才发展伙伴，为科技研发型企业及各类组织提供系统性、分层次、重实战的人才培养解决方案，同时搭建行业前沿技术交流平台，致力于成为企业数字化转型进程中人才培养与业务创新的核心支撑力量，推动组织实现成功转型及经营业绩持续增长。

同时，智盟发起并组织三大 IT 领域品牌会议：QECon 软件质量效能大会、SECon+AgentX 智能时代软件工程技术大会、PMxAI 产品力领航者大会、Agentic AICon 智能体应用与架构工程大会，助力产研中心深化数字化转型、聚焦业务价值达成，在产品研发的快速交付与高质量发展之间找到平衡，打通企业全维度学习视角，最终实现人才能力提升与企业竞争优势增强的双重目标。

详情咨询: 010-65798049

多元化学习形式

技术峰会 → 公开课 → 内训定制 → 微咨询 → 在线课程

核心培训方向

AI大模型应用
产品运营提升
数据分析决策
通用管理赋能
AI职场综合应用

软件研发赋能

大模型前沿	架构设计	数据库	产品经理	持续运维	质量管理
移动开发	组织管理	云计算	开发编程	敏捷开发	大数据

Agentic AICon

统一抽象与分形协作：OxyGent 多智能体架构的设计范式与工程取舍

李天龙 | 京东零售 OxyGent 开源项目架构师

Agentic AICon

李天龙

京东 算法工程师

OxyGent 开源项目架构师

AI基础架构与多智能体领域一线实践专家

目录

01 / 多智能体的架构范式

02 / 工具规模化

03 / 系统级设计

04 / Agent 自主性设计

PART ONE

多智能体的架构范式

01

上下文减法与分形协作——给 LLM 的上下文做减法，让减法在代码里长成分形。

单Agent，扛不住复杂任务

Agentic AICon

1

上下文爆炸

工具 schema、多轮历史、检索结果全挤进一个 Prompt，窗口迅速被撑满

2

职责纠缠

规划、执行、校验揉在一个 Agent，既当选手又当裁判，改一处崩三处

3

过程失控

子任务的前置数据准备、后置结果校验，只能靠 LLM 临场决定，没法固化成可控步骤

单 Agent 适合 Demo 与轻量场景，扛不住企业级复杂任务。

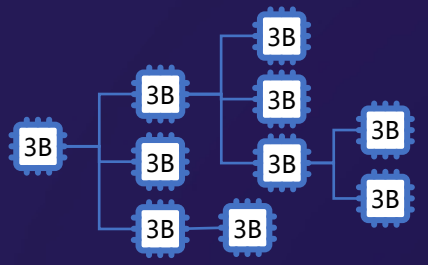
让AI从「单核CPU」进化成「计算集群」

Agentic AICon

让AI从「单核CPU」进化成「计算集群」

Agentic AICon

VS



Single-Agent

Hierarchical Multi-Agent

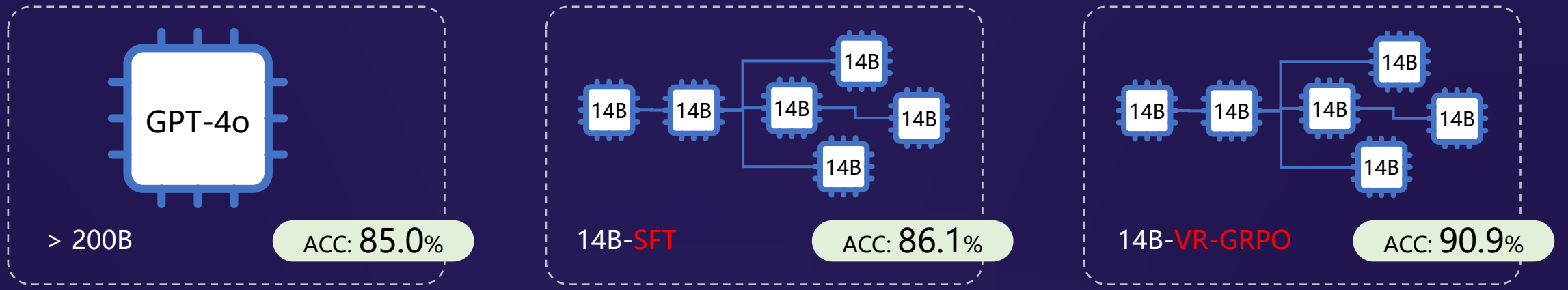
动态规划能力 ↑

容错率 ↑

推理成本 ↓

让AI从「单核CPU」进化成「计算集群」

Agentic AICon

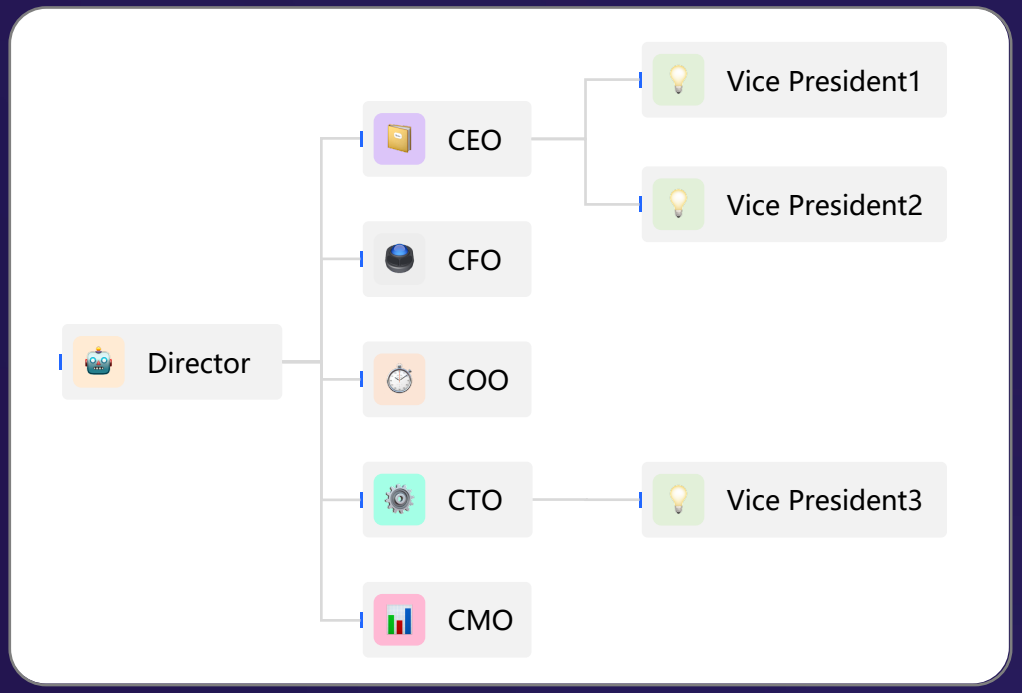


测试集：京麦落地业务

设计哲学：借鉴管理架构与分形几何

Agentic AICon

解决职责边界



像公司分部门：总监 → 经理 → 员工，逐层委派

自闭环的业务部门 + 专注的中台部门

每层只管自己的部分：向上负责、向下调度

解决无限组合

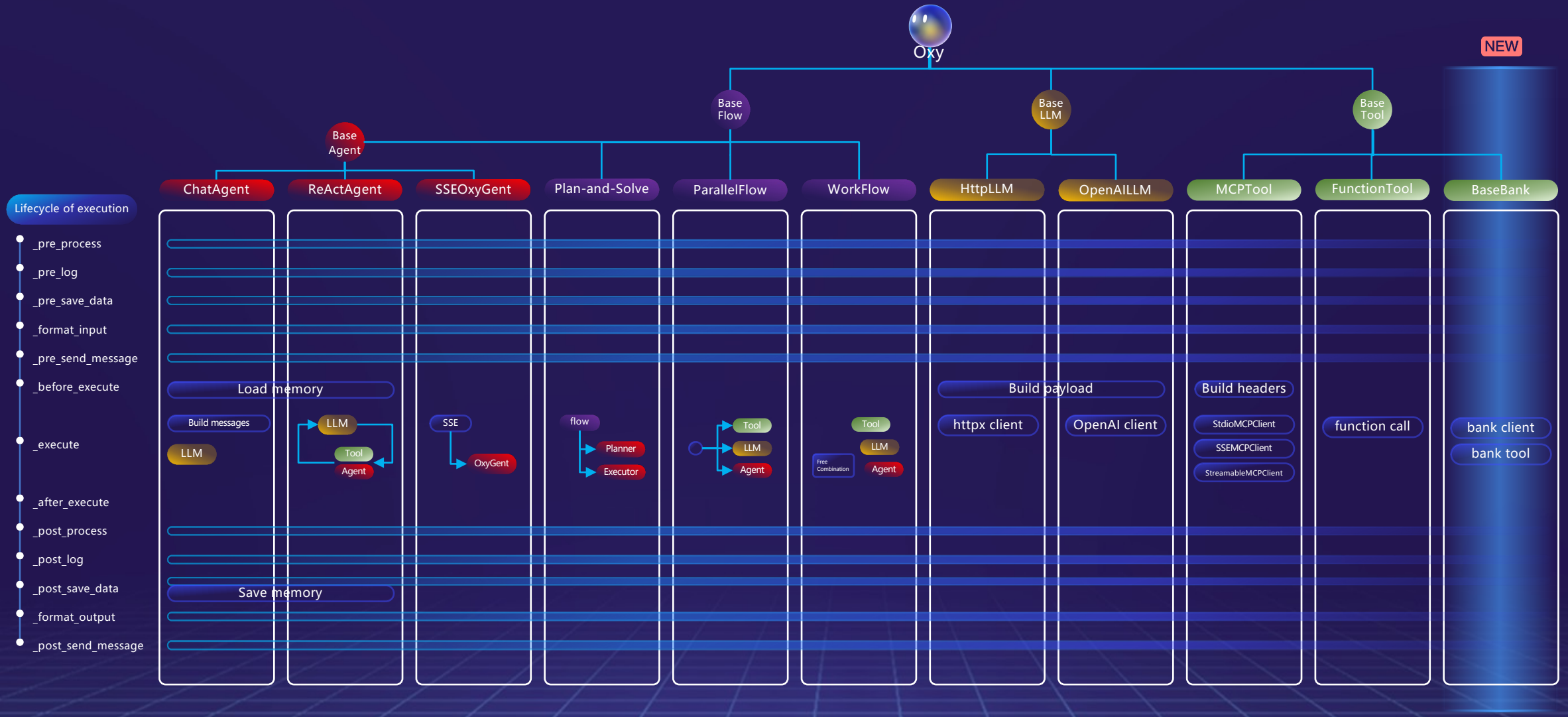
整体的局部，还是整体（自相似）

Agent 调 Agent，多个 Agent 又构成一个 Agent

同一种结构，在每一层重复

一切皆 Oxy

Agentic AICon



一切皆 Oxy：四个工程红利

Agentic AICon

01

自由编排

Agent 当工具调、编排像搭积木；切换编排模式改动很小。

02

横切能力一处实现

trace、日志、权限、超时、缓存、重试在同一套生命周期里统一做，所有节点自动生效、全链路一致。

03

可轻可重

一个 Oxy 可以是一次 LLM 调用，也可以是完整 ReAct Agent，按需取用。

04

易扩展

新组件只要实现 Oxy 接口就能接入编排，编排层样板代码约 −70%。

统一抽象的价值 > 单组件的灵活性。

像搭积木构建多智能体系统

Agentic AICon

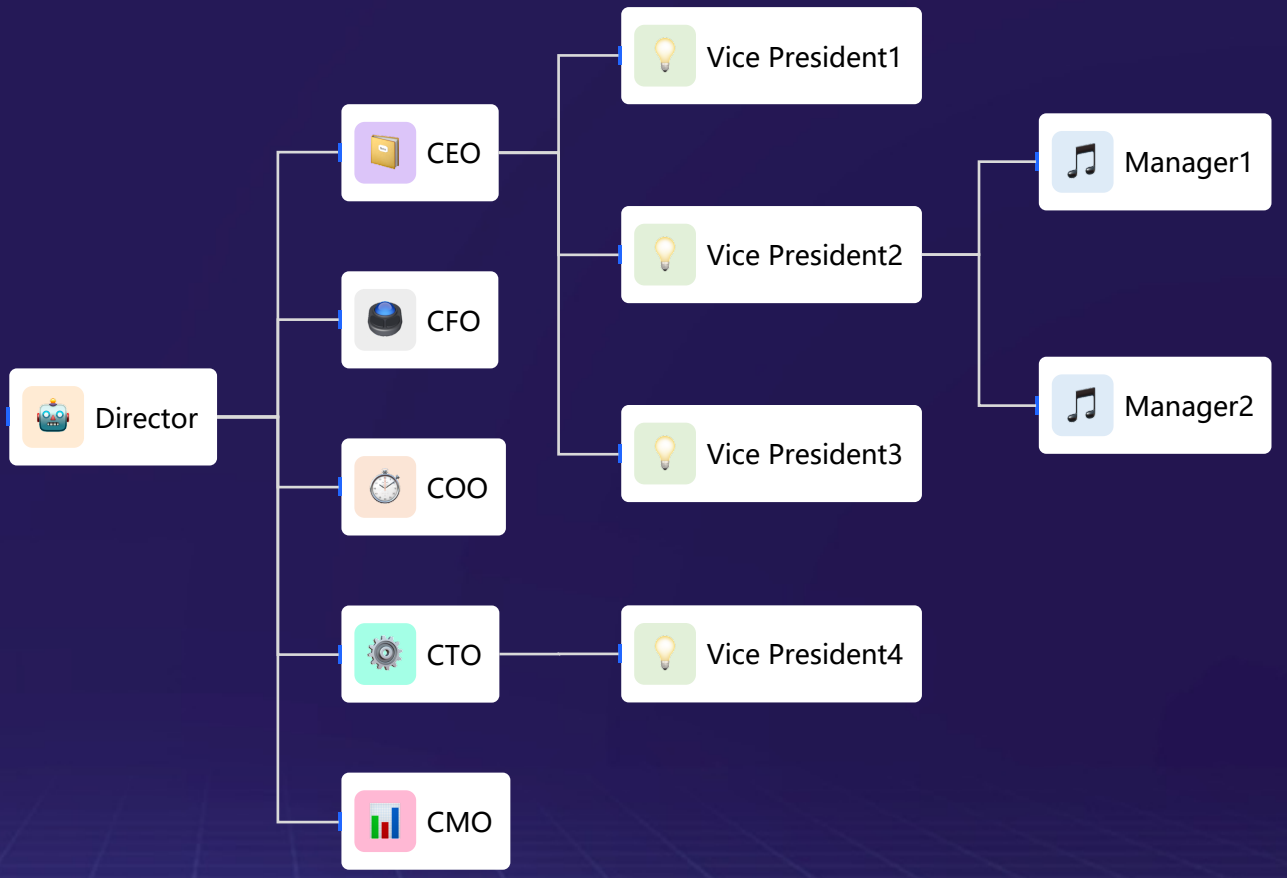
```python
import os
from oxygent import MAS, oxy

def load_agent_infos():…

oxy_space = [
    oxy.HttpLLM(
        name="default_llm",
        api_key=os.getenv("DEFAULT_LLM_API_KEY"),
        base_url=os.getenv("DEFAULT_LLM_BASE_URL"),
        model_name=os.getenv("DEFAULT_LLM_MODEL_NAME"),
    ),
]
for agent_info in load_agent_infos():
    oxy_space.append(
        oxy.ReActAgent(
            name=agent_info.name,
            desc=agent_info.desc,
            tools=agent_info.tools,
            sub_agents=agent_info.sub_agents,
            llm_model="default_llm",
        ),
    )

async def main():
    async with MAS(oxy_space=oxy_space) as mas:
        await mas.start_web_service()

if __name__ == "__main__":
    import asyncio
    asyncio.run(main())
```



5 分钟 2000+ 智能体

多Agent按上下文拆，而不是按环节拆

Agentic AICon

× 按环节拆

规划 Agent
需要懂 订单 · 促销 · 财务 · 售后

↓ 携带一大堆 上下文

执行 Agent
需要懂 订单 · 促销 · 财务 · 售后

↓ 携带一大堆 上下文

校验 Agent
需要懂 订单 · 促销 · 财务 · 售后

每个 Agent 都横跨全部领域 →
上下文没减少，还在 Agent 间反复搬运

✓ 按上下文拆

订单 Agent
订单查询、订单报备

营销 Agent
活动创建、活动审核

财务 Agent
对账、结算

售后 Agent
退换货、投诉

每个 Agent 只装自己领域、本地闭环 →
上下文干净、选得准

同样是拆成多智能体，按功能拆每个 Agent 还得懂所有领域；按上下文拆每个 Agent 只装自己一个领域。

PART TWO

工具规模化

02

三种协同架构模式——逆向规划、向量召回、多层路由，从源头减 N、选得准、分层治 N。

候选一多，纯 LLM 调度就崩

塞全

所有工具描述进 prompt
→ **上下文爆炸，又贵又慢**

不塞全

模型**不知道有这个工具**，直接漏掉

别急着换更强的模型

不是模型不行，是「让模型在过多候选里选」这件事本身不成立。

给你 100 个按钮、没分类、没搜索——人也会按错。

所以解法不是把模型练更强，而是 别让它在过多候选里硬选 —— **减少候选 / 优化语料 / 分层**。

从「减 N」到「选得准」到「分层治 N」

Agentic AICon

减N

方案一 · 治本

工具治理 · 逆向规划

从目标倒推真正需要的工具集，治理冗余，把 N 从源头降下来

候选变少

选得准

方案二 · 通用兜底

向量召回 · 语料优化

向量召回缩到 Top-K（约10个）直接交 Agent 选；语料由 OxySourcing 持续评测、调优。

OxySourcing

分层治N

方案三 · 分层

多层 Agent 调度

不同工具交给不同 Agent、Agent 可多层，每层只在少数候选里选。

按上下文拆

三招通常是**组合拳** —— 先减 N，再优化语料，最后按域分层。

从「减 N」到「选得准」到「分层治 N」

Agentic AICon

减N

方案一 · 治本

工具治理 · 逆向规划

从目标倒推真正需要的工具集，治理冗余，把 N 从源头降下来

候选变少

选得准

方案二 · 通用兜底

向量召回 · 语料优化

向量召回缩到 Top-K（约10个）直接交 Agent 选；语料由 OxySourcing 持续评测、调优。

OxySourcing

分层治N

方案三 · 分层

多层 Agent 调度

不同工具交给不同 Agent、Agent 可多层，每层只在少数候选里选。

按上下文拆

三招通常是**组合拳** —— 先减 N，再优化语料，最后按域分层。

工具治理 · 逆向规划

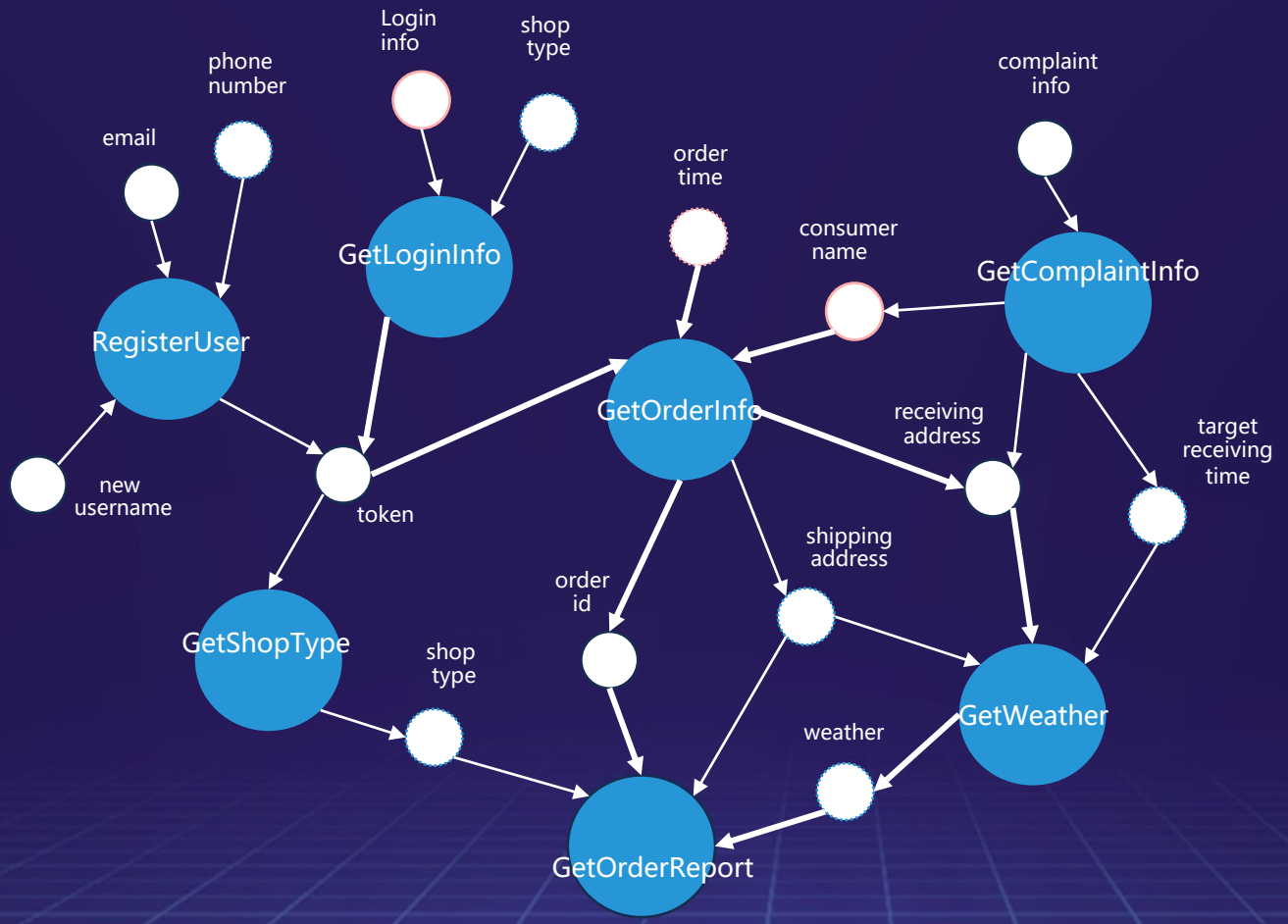
Agentic AICon

彼此独立的工具 → 有关系的工具图 → 逆向规划

工具治理 · 逆向规划

Agentic AICon

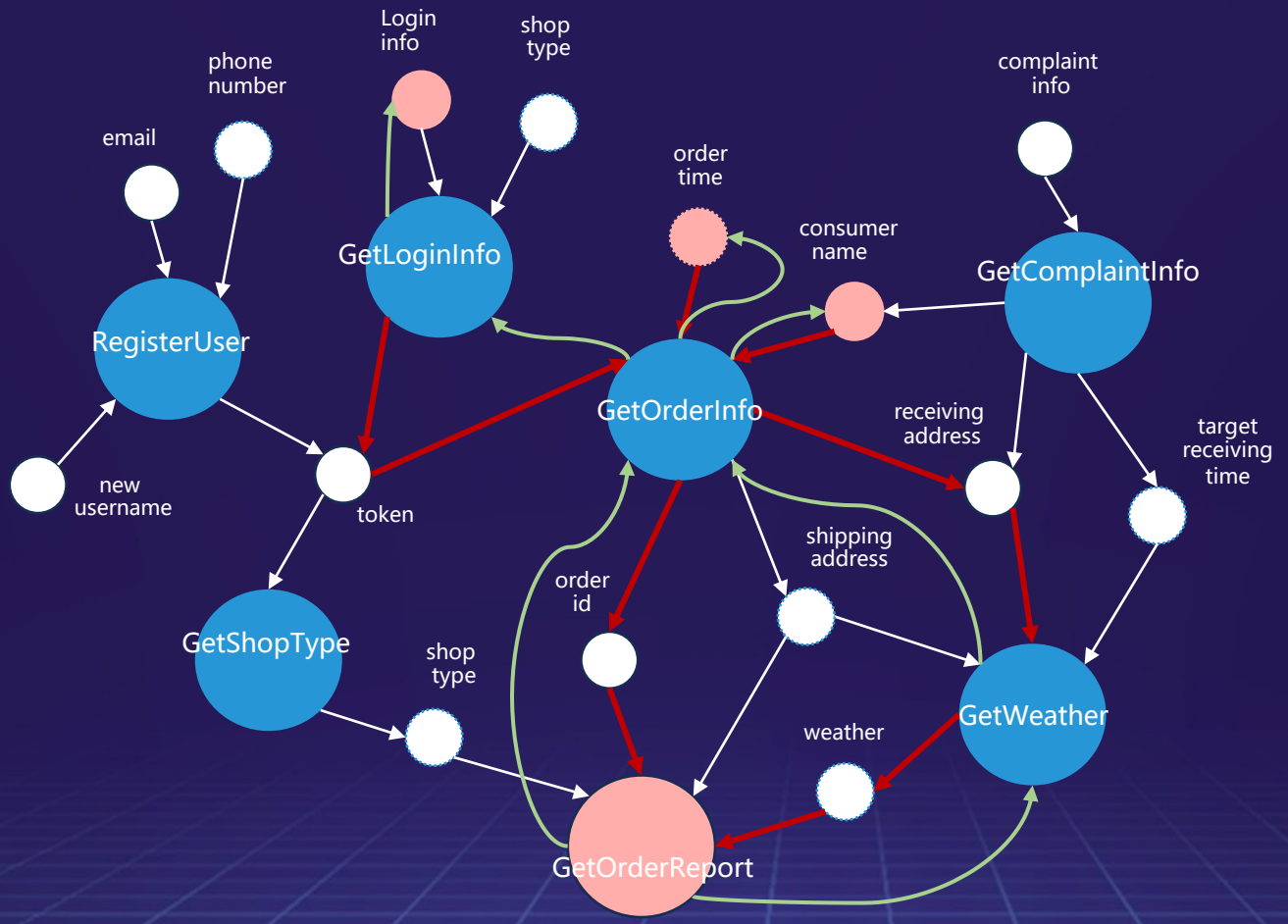
彼此独立的工具 → 有关系的工具图 → 逆向规划



工具治理·逆向规划

Agentic AICon

彼此独立的工具 → 有关系的工具图 → 逆向规划



95%+

B端超级智能体
工具调用准确率

从「减 N」到「选得准」到「分层治 N」

Agentic AICon

减N

方案一 · 治本

工具治理 · 逆向规划

从目标倒推真正需要的工具集，治理冗余，把 N 从源头降下来

候选变少

选得准

方案二 · 通用兜底

向量召回 · 语料优化

向量召回缩到 Top-K（约10个）直接交 Agent 选；语料由 OxySourcing 持续评测、调优。

OxySourcing

分层治N

方案三 · 分层

多层 Agent 调度

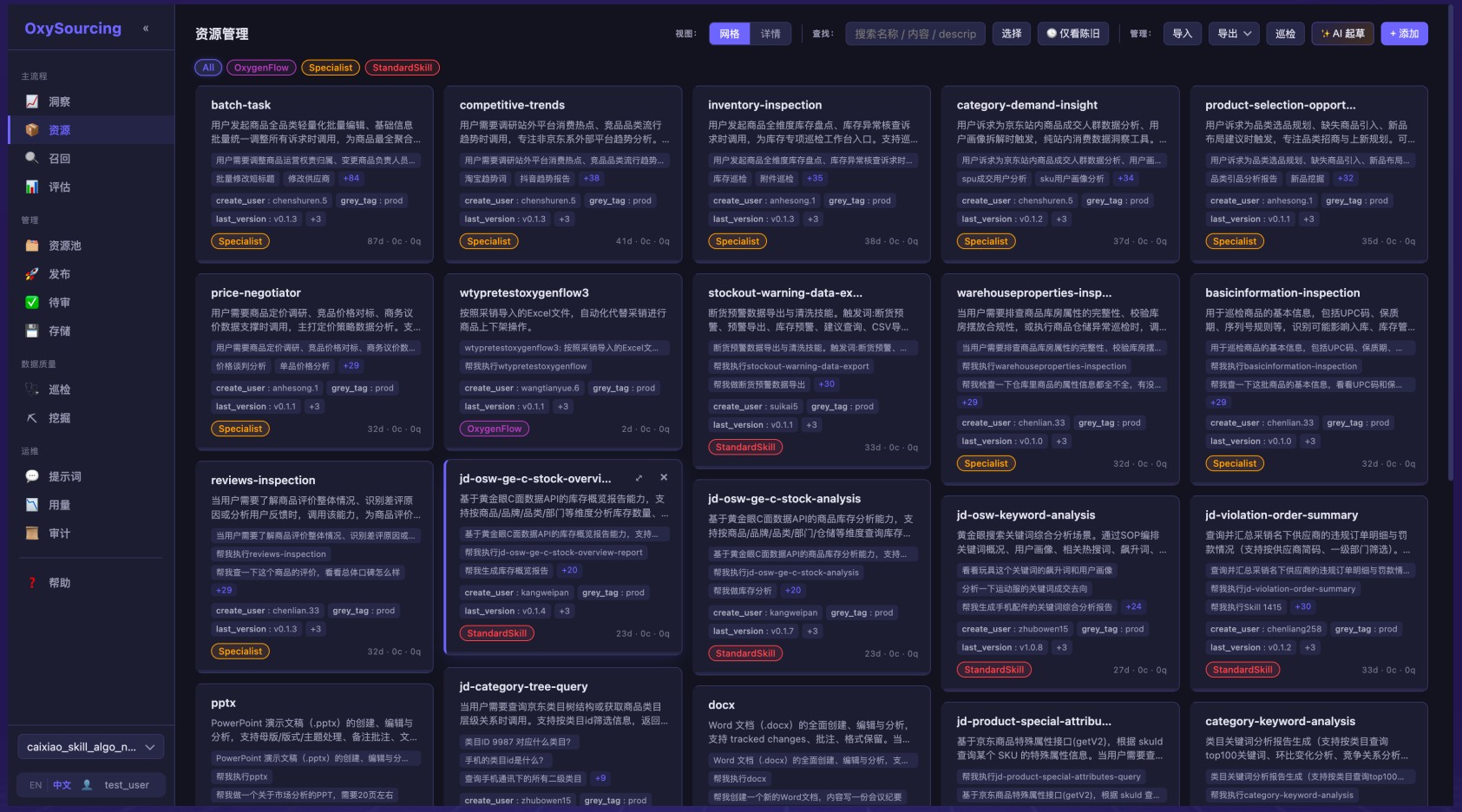
不同工具交给不同 Agent、Agent 可多层，每层只在少数候选里选。

按上下文拆

三招通常是**组合拳** —— 先减 N，再优化语料，最后按域分层。

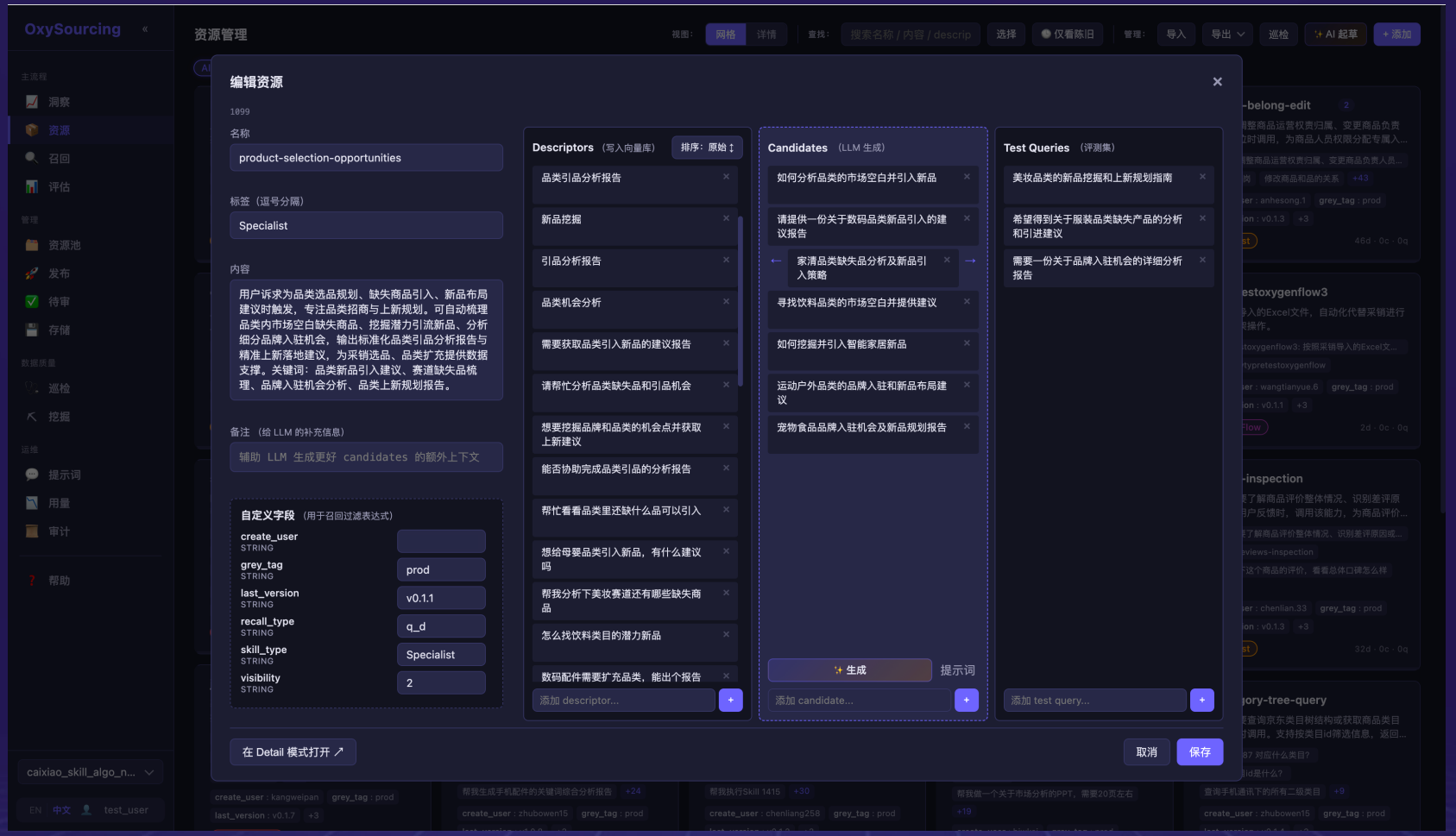
向量召回 · 语料优化

Agentic AICon



Tool、Agent、Skill 被统一定义为可召回的资源

向量召回 · 语料优化



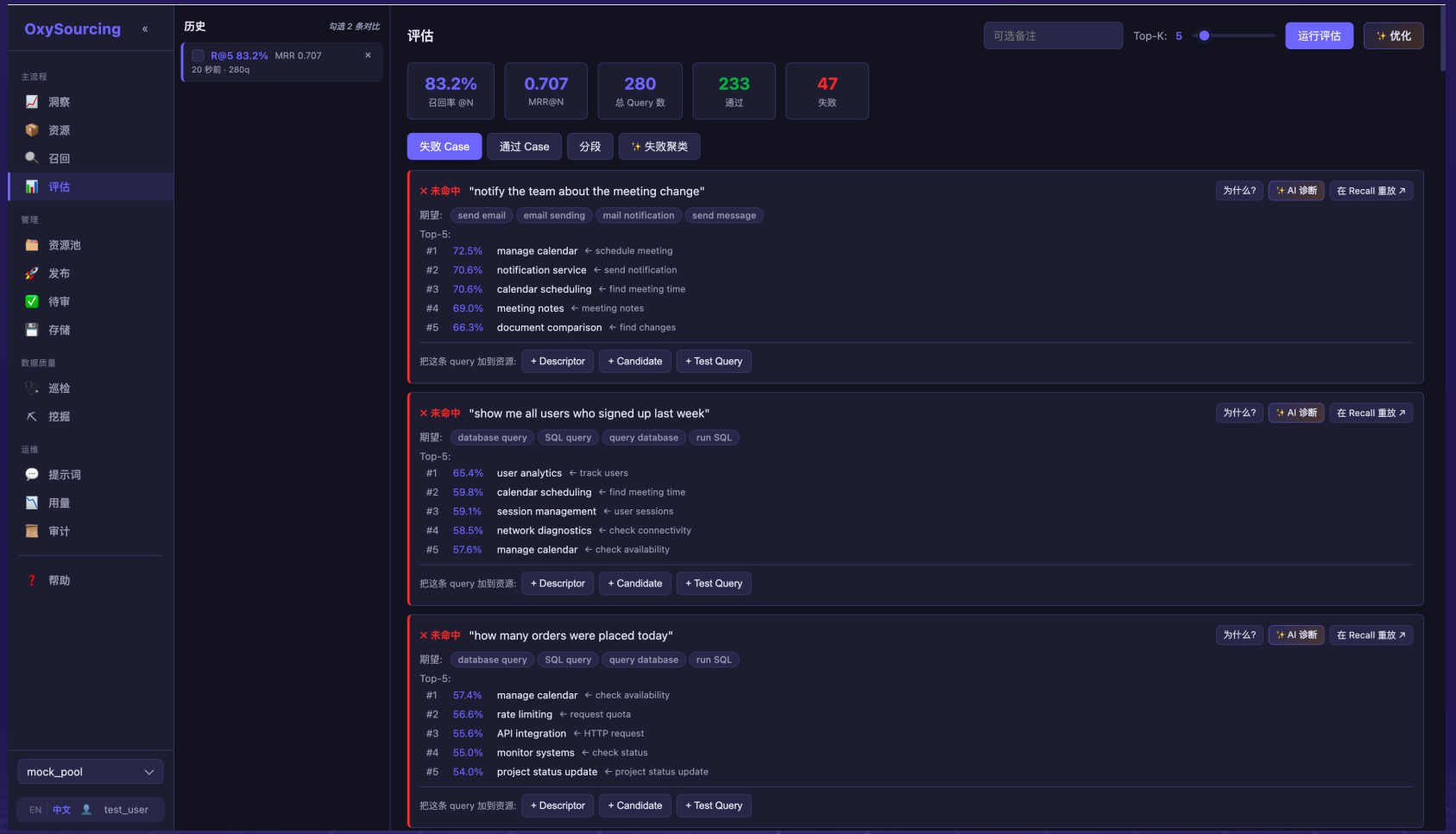
每个资源同时维护：描述、候选、测试 三个集合

向量召回 · 语料优化

二维可视化展示资源分布

向量召回 · 语料优化

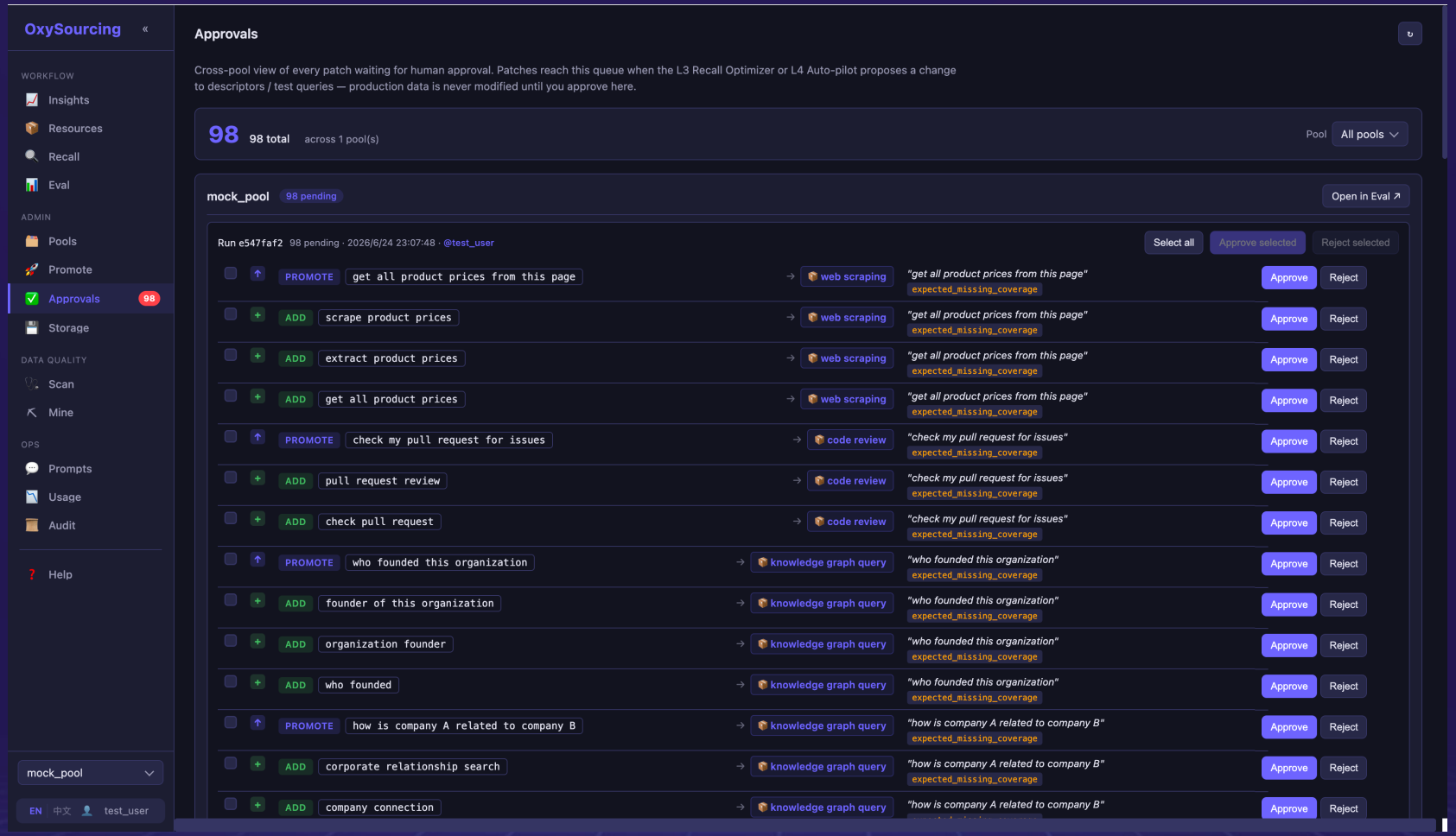
Agentic AICon



随时评估召回准确率

向量召回 · 语料优化

Agentic AICon



根据线上真实query智能增删改语料

从「减 N」到「选得准」到「分层治 N」

Agentic AICon

减N

方案一 · 治本

工具治理 · 逆向规划

从目标倒推真正需要的工具集，治理冗余，把 N 从源头降下来

候选变少

选得准

方案二 · 通用兜底

向量召回 · 语料优化

向量召回缩到 Top-K（约10个）直接交 Agent 选；语料由 OxySourcing 持续评测、调优。

OxySourcing

分层治N

方案三 · 分层

多层 Agent 调度

不同工具交给不同 Agent、Agent 可多层，每层只在少数候选里选。

按上下文拆

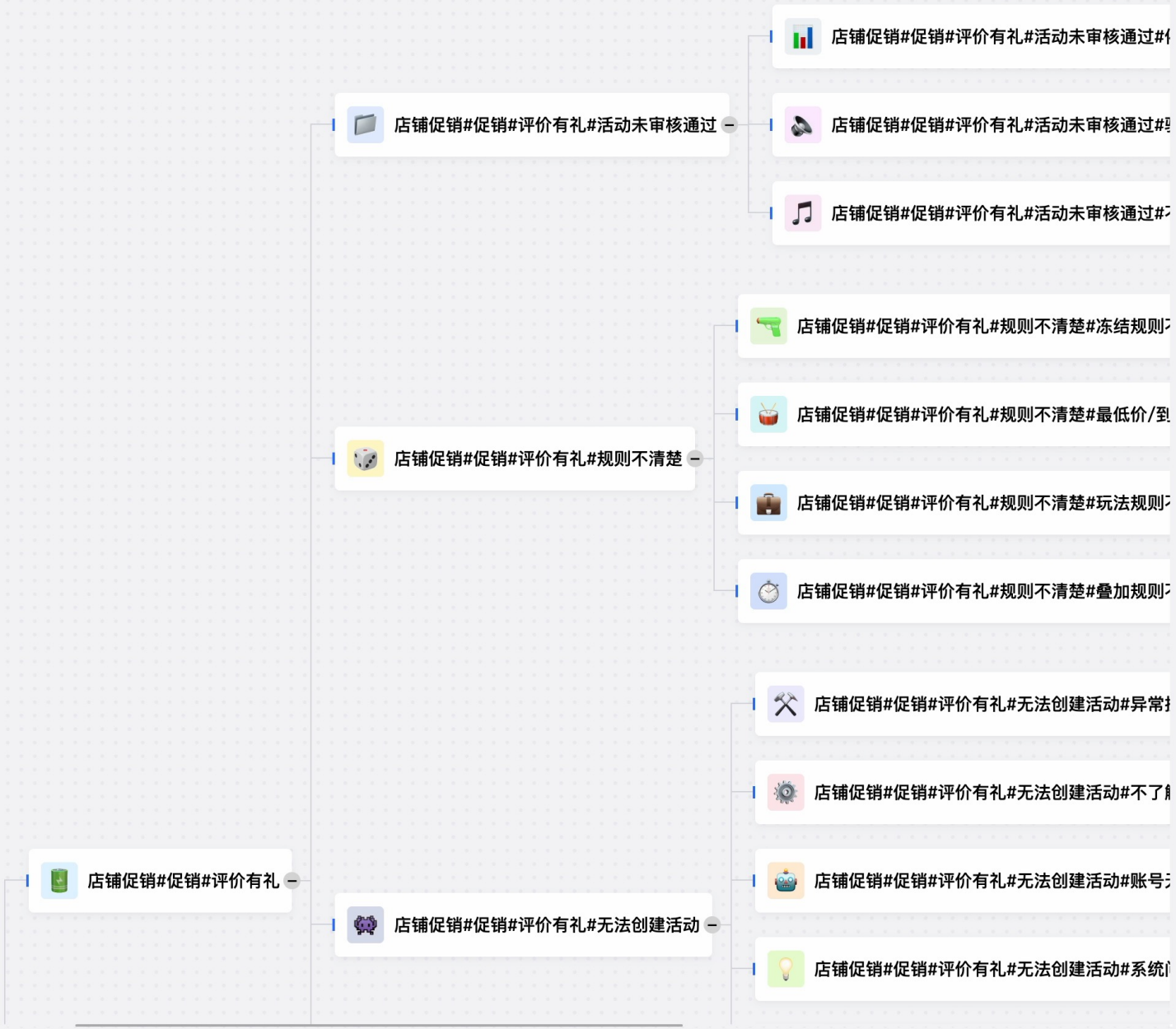
三招通常是**组合拳** —— 先减 N，再优化语料，最后按域分层。

多层 Agent 调度：逐层分诊，每层仅少数候选

Agentic AICon

用户之声——把成千上万条客户声音，分发到多层的业务域。

不是从 5×3×4 = 60 个叶子里选一次（崩），而是**分多层、每层 3~5 选 1**（稳）。



Playground

master_agent

How may I support your request today?

<顾客>:以下为商家近5min与智能客服的对话问题：问题1：人工问题2：人工|<顾客>:你好|<

PART THREE

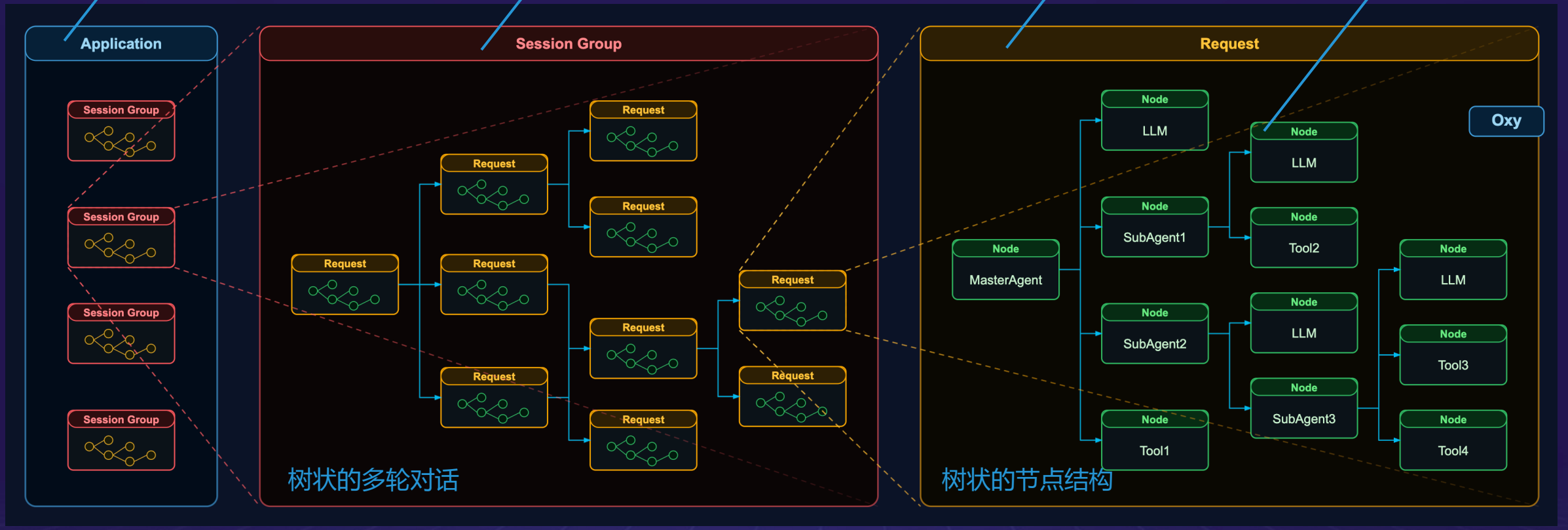
系统级设计

03

可扩展性、容错性与可观测性——用作用域、强类型契约与全链路 Trace，把 Agent 从 Demo 推到生产。

OxyGent 资源管理与数据共享

Agentic AICon



四大数据作用域，通过上下文对象访问全域数据

细粒度会话控制

Agentic AICon

树状的多轮对话 >>> 新的产品形态设计

QuickStart

Agentic AICon

- Step1:
pip install oxygent

- Step2:
Edit .env

- Step3:
Edit demo.py

- Step4:
python demo.py

```python
import os
from oxygent import MAS, oxy

oxy_space = [
    oxy.HttpLLM(
        name="default_llm",
        api_key=os.getenv("DEFAULT_LLM_API_KEY"),
        base_url=os.getenv("DEFAULT_LLM_BASE_URL"),
        model_name=os.getenv("DEFAULT_LLM_MODEL_NAME"),
    ),
    oxy.StdioMCPClient(
        name="time_tools",
        params={"command": "uvx",
                "args": ["mcp-server-time", "--local-timezone=Asia/Shanghai"]}
    ),
    oxy.StdioMCPClient(
        name="file_tools",
        params={"command": "npx",
                "args": ["-y", "@modelcontextprotocol/server-filesystem", "./"]}
    ),
    oxy.ReActAgent(
        name="time_agent",
        desc="一个可以查询时间的工具",
        tools=["time_tools"],
        llm_model="default_llm",
    ),
    oxy.ReActAgent(
        name="file_agent",
        desc="一个可以操作文件系统的工具",
        tools=["file_tools"],
        llm_model="default_llm",
    ),
    oxy.ReActAgent(
        is_master=True,
        name="master_agent",
        sub_agents=["time_agent", "file_agent"],
        llm_model="default_llm",
    ),
]

函数注释 | 代码评审 | 代码重构 | 逐行注释 | 单元测试 | ×
async def main():
    async with MAS(oxy_space=oxy_space) as mas:
        await mas.start_web_service(first_query="现在几点，记到time.txt里")

if __name__ == "__main__":
    import asyncio
    asyncio.run(main())
```

QuickStart

- Step1:
pip install oxygent

- Step2:
Edit .env

- Step3:
Edit demo.py

- Step4:
python demo.py

```python
import os
from oxygent import MAS, oxy

oxy_space = [
    oxy.HttpLLM(
        name="default_llm",
        api_key=os.getenv("DEFAULT_LLM_API_KEY"),
        base_url=os.getenv("DEFAULT_LLM_BASE_URL"),
        model_name=os.getenv("DEFAULT_LLM_MODEL_NAME"),
    ),
        params={"command": "npx",
                "args": ["-y", "@modelcontextprotocol/server-filesystem", "./"]}
    ),
    oxy.ReActAgent(
        name="time_agent",
        desc="一个可以查询时间的工具",
        tools=["time_tools"],
        llm_model="default_llm",
    ),
    oxy.ReActAgent(
        name="file_agent",
        desc="一个可以操作文件系统的工具",
        tools=["file_tools"],
        llm_model="default_llm",
    ),
    oxy.ReActAgent(
        is_master=True,
        name="master_agent",
        sub_agents=["time_agent", "file_agent"],
        llm_model="default_llm",
    ),
]

函数注释 | 代码评审 | 代码重构 | 逐行注释 | 单元测试 | ×
async def main():
    async with MAS(oxy_space=oxy_space) as mas:
        await mas.start_web_service(first_query="现在几点，记到time.txt里")

if __name__ == "__main__":
    import asyncio
    asyncio.run(main())
```

注册大模型

QuickStart

Agentic AICon

- Step1:
pip install oxygent

- Step2:
Edit .env

- Step3:
Edit demo.py

- Step4:
python demo.py

```python
import os
from oxygent import MAS, oxy

oxy_space = [
    oxy.HttpLLM(
        name="default_llm",
        api_key=os.getenv("DEFAULT_LLM_API_KEY"),
        base_url=os.getenv("DEFAULT_LLM_BASE_URL"),
        llm_model="default_llm",
    ),
    oxy.ReActAgent(
        is_master=True,
        name="master_agent",
        sub_agents=["time_agent", "file_agent"],
        llm_model="default_llm",
    ),
]

async def main():
    async with MAS(oxy_space=oxy_space) as mas:
        await mas.start_web_service(first_query="现在几点，记到time.txt里")

if __name__ == "__main__":
    import asyncio
    asyncio.run(main())
```

注册MCP工具

```python
oxy.StdioMCPClient(
    name="time_tools",
    params={"command": "uvx",
            "args": ["mcp-server-time", "--local-timezone=Asia/Shanghai"]}
),
oxy.StdioMCPClient(
    name="file_tools",
    params={"command": "npx",
            "args": ["-y", "@modelcontextprotocol/server-filesystem", "./"]}
),
```

QuickStart

Agentic AICon

- Step1:
pip install oxygent

- Step2:
Edit .env

- Step3:
Edit demo.py

- Step4:
python demo.py

```python
import os
from oxygent import MAS, oxy

oxy_space = [
    oxy.ReActAgent(
        name="time_agent",
        desc="一个可以查询时间的工具",
        tools=["time_tools"],
        llm_model="default_llm",
    ),
    oxy.ReActAgent(
        name="file_agent",
        desc="一个可以操作文件系统的工具",
        tools=["file_tools"],
        llm_model="default_llm",
    ),
    oxy.ReActAgent(
        is_master=True,
        name="master_agent",
        sub_agents=["time_agent", "file_agent"],
        llm_model="default_llm",
    ),
]

函数
async def main():
    async with MAS(oxy_space=oxy_space) as mas:
        await mas.start_web_service(first_query="现在几点，记到time.txt里")

if __name__ == "__main__":
    import asyncio
    asyncio.run(main())
```

注册智能体
并标识权限关系

QuickStart

Agentic AICon

- Step1:
pip install oxygent

- Step2:
Edit .env

- Step3:
Edit demo.py

- Step4:
python demo.py

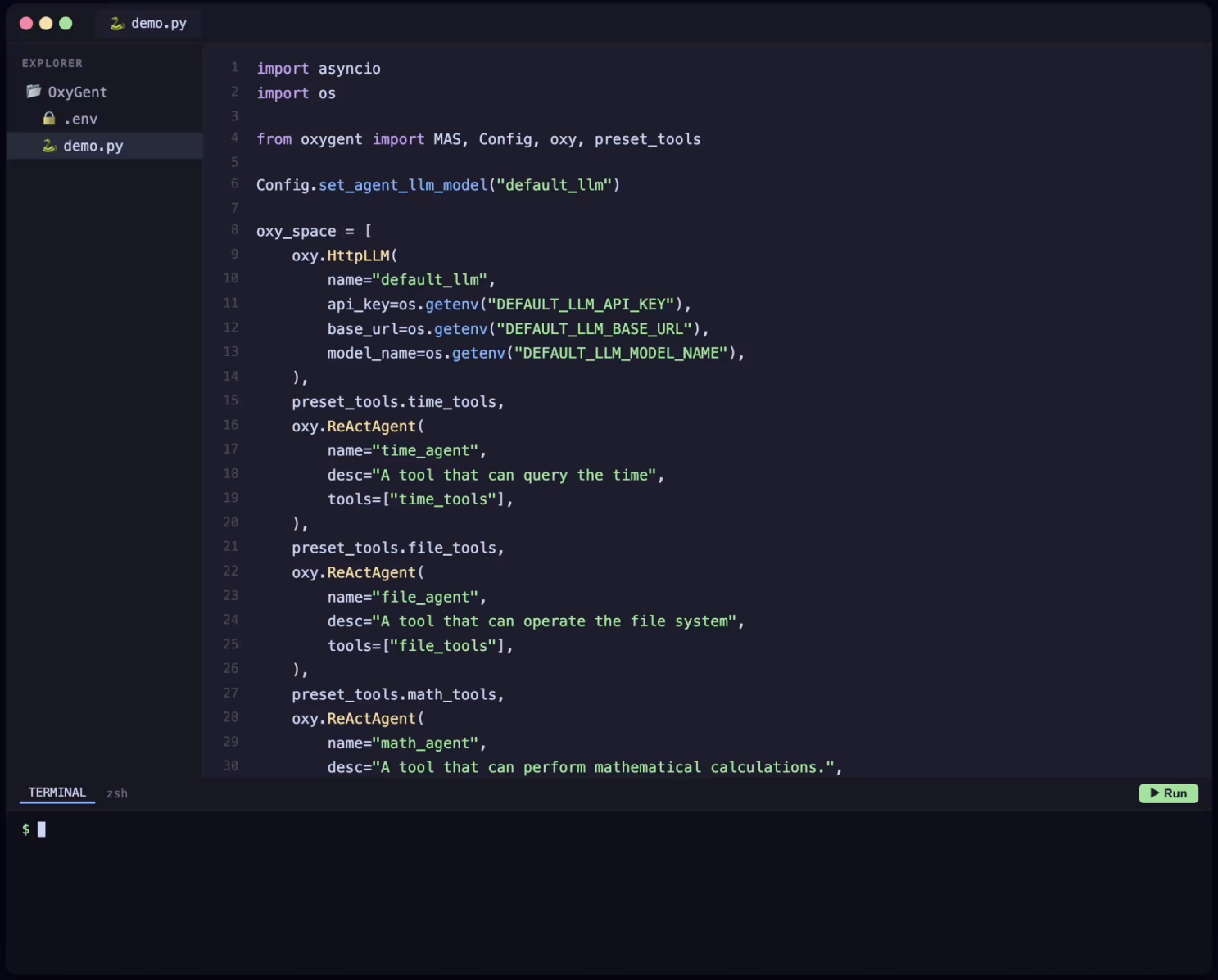
```python
import os
from oxygent import MAS, oxy

oxy_space = [
    oxy.HttpLLM(
        name="default_llm",
        api_key=os.getenv("DEFAULT_LLM_API_KEY"),
        base_url=os.getenv("DEFAULT_LLM_BASE_URL"),
        model_name=os.getenv("DEFAULT_LLM_MODEL_NAME"),
    ),
    oxy.StdioMCPClient(
        name="time_tools",
        params={"command": "uvx",
                "args": ["mcp-server-time", "--local-timezone=Asia/Shanghai"]}
    ),
    oxy.StdioMCPClient(
        name="file_tools",
        params={"command": "npx",
                "args": ["-y", "@modelcontextprotocol/server-filesystem", "./"]}
    ),
    oxy.ReActAgent(
        name="time_agent",
        desc="一个可以查询时间的工具",
        tools=["time_tools"],
        llm_model="default_llm",
    ),
    oxy.ReActAgent(
        name="file_agent",
        desc="一个可以操作文件系统的工具",
        tools=["file_tools"],
        llm_model="default_llm",
    ),
    oxy.ReActAgent(
        is_master=True,
        name="master_agent",
        sub_agents=["time_agent", "file_agent"],
        llm_model="default_llm",
    )
]

async def main():
    async with MAS(oxy_space=oxy_space) as mas:
        await mas.start_web_service(first_query="现在几点，记到time.txt里")

if __name__ == "__main__":
    import asyncio
    asyncio.run(main())
```

启动MAS服务


demo.py
EXPLORER
OxyGent
.env
demo.py
import asyncio
import os
from oxygent import MAS, Config, oxy, preset_tools
Config.set_agent_llm_model("default_llm")
oxy_space = [
oxy.HttpLLM(
name="default_llm",
api_key=os.getenv("DEFAULT_LLM_API_KEY"),
base_url=os.getenv("DEFAULT_LLM_BASE_URL"),
model_name=os.getenv("DEFAULT_LLM_MODEL_NAME"),
),
preset_tools.time_tools,
oxy.ReActAgent(
name="time_agent",
desc="A tool that can query the time",
tools=["time_tools"],
),
preset_tools.file_tools,
oxy.ReActAgent(
name="file_agent",
desc="A tool that can operate the file system",
tools=["file_tools"],
),
preset_tools.math_tools,
oxy.ReActAgent(
name="math_agent",
desc="A tool that can perform mathematical calculations.",
TERMINAL
zsh
Run
$

企业级多智能体系统

Agentic AICon

知识库平台
- KB Server1
- KB Server2

MCP平台
- MCP Server1
- MCP Server2

LLM平台
- DeepSeek Server
- GPT Server
- LLM weight

Agent广场
- OxyGent Server
- A2A Server

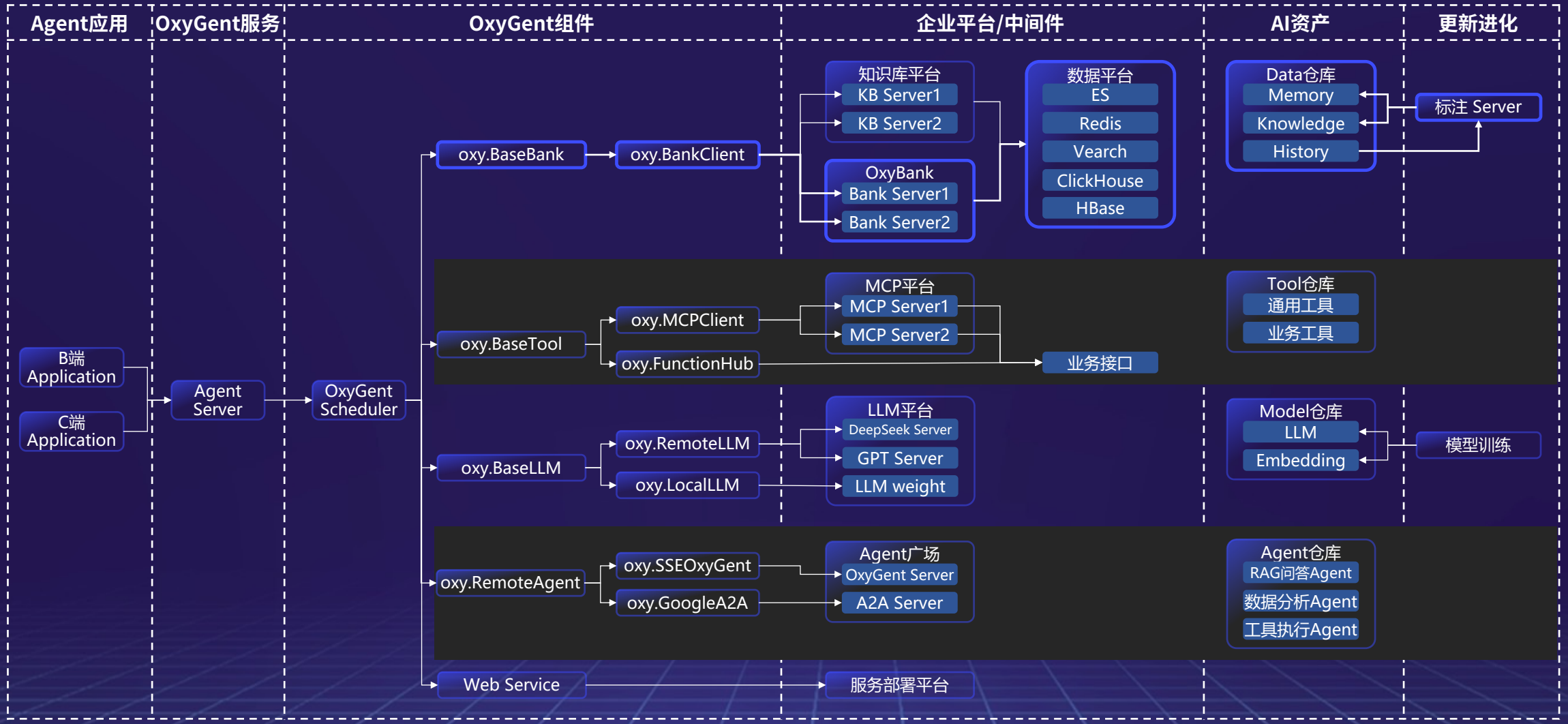
数据平台
- ES
- Redis
- Vearch
- ClickHouse
- HBase

服务监控平台

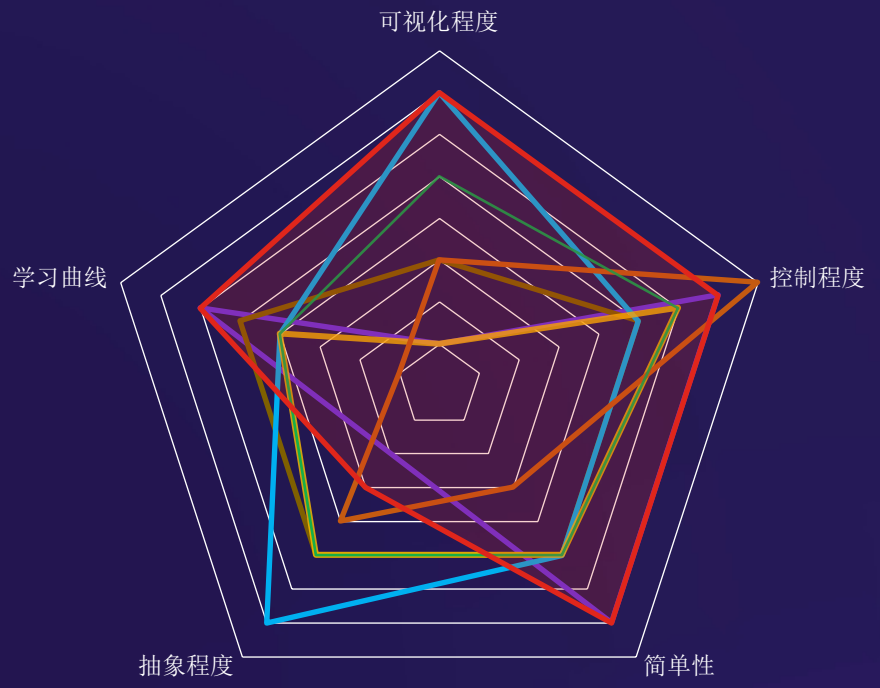
服务部署平台

OxyGent —— 连接企业平台/中间件与Agent的桥梁

Agentic AICon



OxyGent 对标行业



OpenAI Agents SDK	OpenAI	以Python为主，核心原语（Agents、Handoffs、Guardrails），直接控制
CrewAI	CrewAI	基于角色的代理、团队、任务，注重协作
AutoGen	Microsoft	会话型Agent，灵活的对话模式，支持人类参与
Google ADK	Google	多Agent分层结构，集成 Google Cloud（Gemini、Vertex AI），丰富工具生态，双向流
LangGraph	LangChain	基于图的工作流、节点、边、显式状态管理
Dapr Agents	Microsoft	有状态虚拟参与者，事件驱动多Agent工作流，Kubernetes 集成，50+数据连接器
OxyGent	JD	协议无关，插件化扩展，横切能力原生支持

欢迎加入 OxyGent 社区

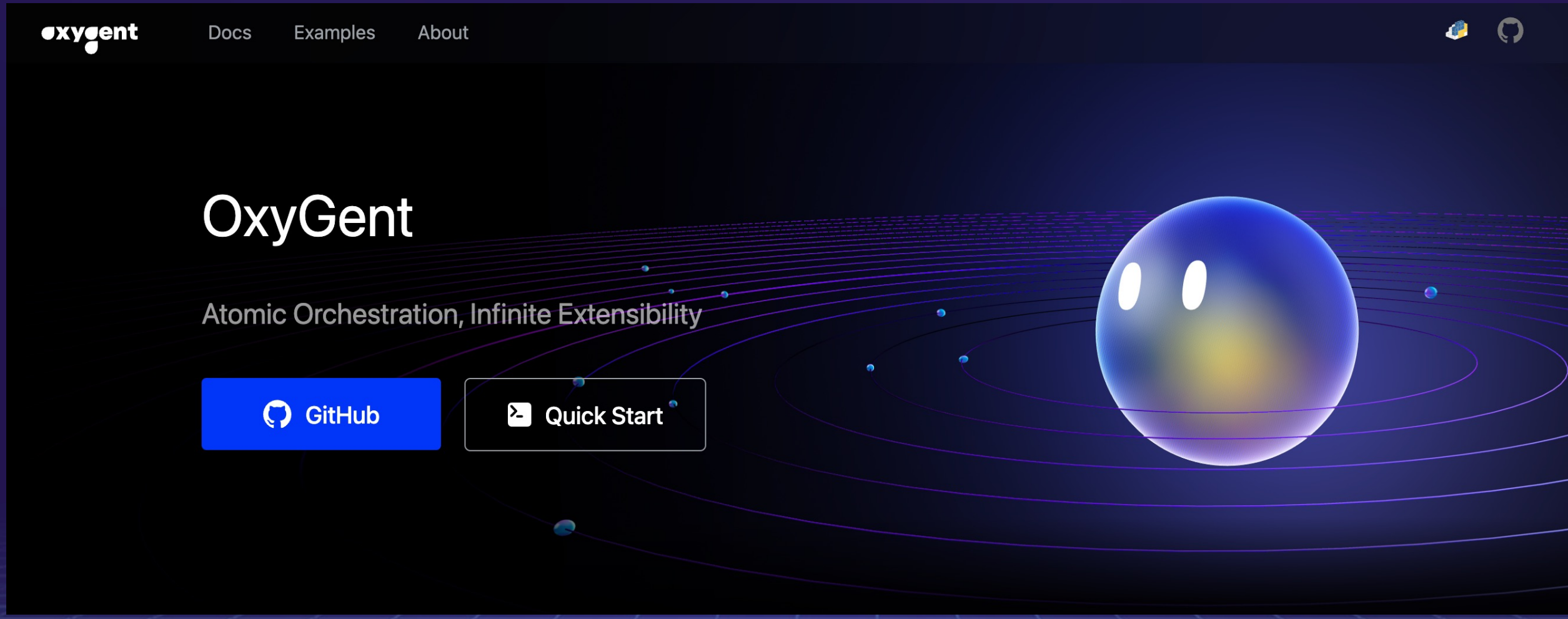
Agentic AICon

https://oxygent.jd.com

官网二维码

欢迎加入 OxyGent 社区

Agentic AICon



PART FOUR

Agent 自主性设计

04

L0–L4 分级与落地——先回答委托、边界、兜底三问，再按等级逐级上移。

Agent落地第一步是明确自主的边界

Agentic AICon

它决定了后续所有的架构设计。先回答三个问题：

Q1 委托范围

哪些任务可以全权委托给 Agent?

划定 Agent 能独立闭环的边界

Q2 人机边界

哪些决策必须由人类确认?

高风险动作留人工卡点

Q3 失败兜底

决策失误时的兜底与回退是什么?

越界即熔断、可回滚、转人工

用分级把「边界」讲清楚

Agentic AICon

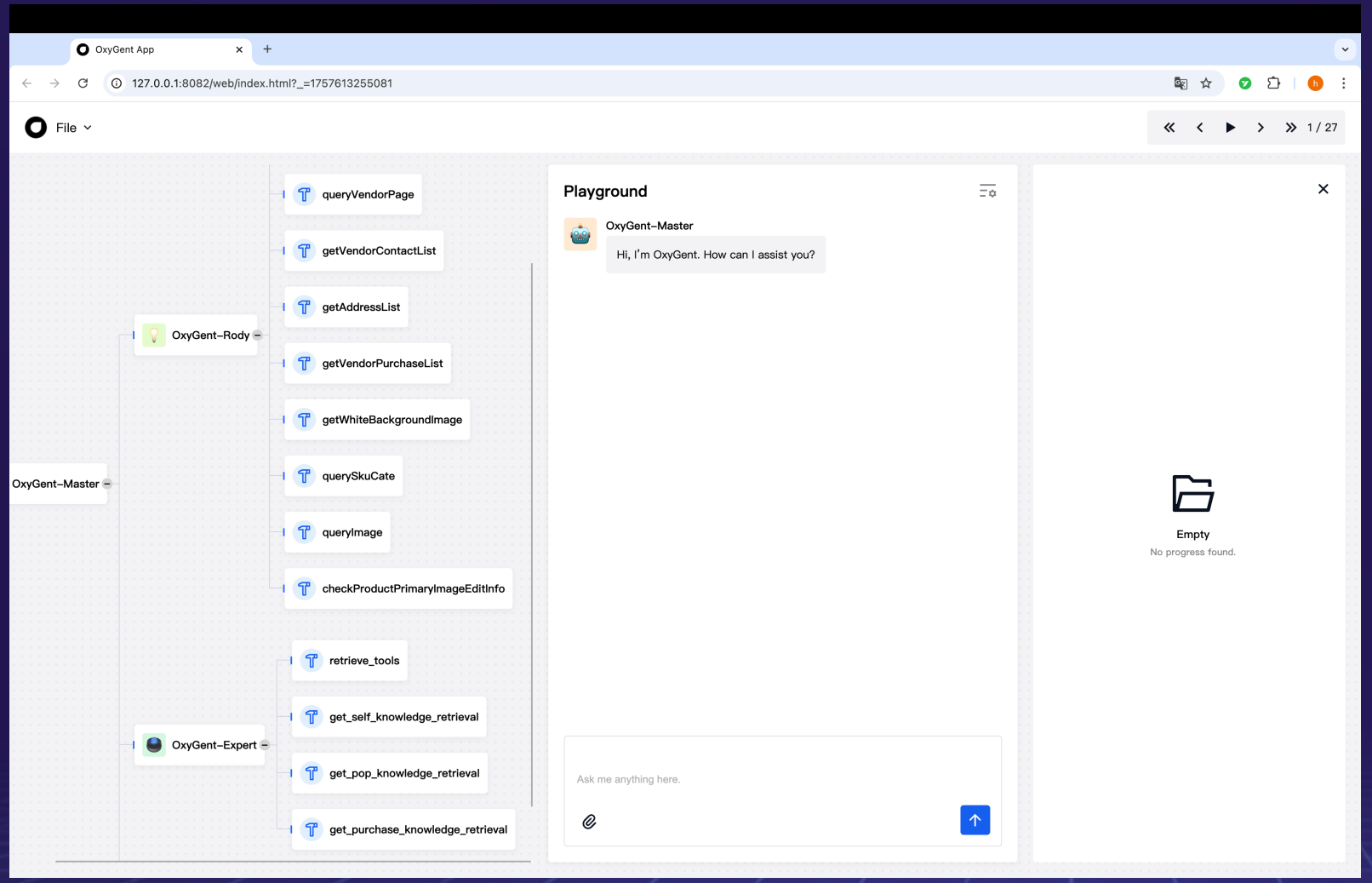
等级	智能形态	人委托给AI的对象	人的核心责任	AI的核心责任	AI执行权限	判定标准
L0	人工操作	不委托	完成全流程生产	提供基础计算/存储工具	无	AI不参与任何决策链路
L1	步骤辅助	某个原子操作	发起任务、串联全局、终审裁决	按指令完成单一环节（如检索、生成、校验）	无	AI输出中间结果，人类手工搬运
L2	任务协同	一个完整闭环任务	下达目标指令、审批最终交付物	独立闭环完成任务（理解意图→生成方案→输出成品）	原则上不直接生效	AI输出最终成品，人类确认后生效
L3	受控托管	一类确定性事务	设定边界规则、异常监控与仲裁	在边界内自主触发、决策并执行，遇边界外转人工	有限执行（带熔断）	AI直接执行常规动作，异常时主动交还控制权
L4	自主运营	一个持续性经营指标	设定指标、资源预算与伦理红线	全周期自主（感知→规划→执行→评估→自我迭代）	持续执行，强制可回滚	AI围绕目标长期自运维，人类仅做周期审计

不是一步冲到 L4 —— 按业务风险选定当前等级，随信任建立逐级上移。

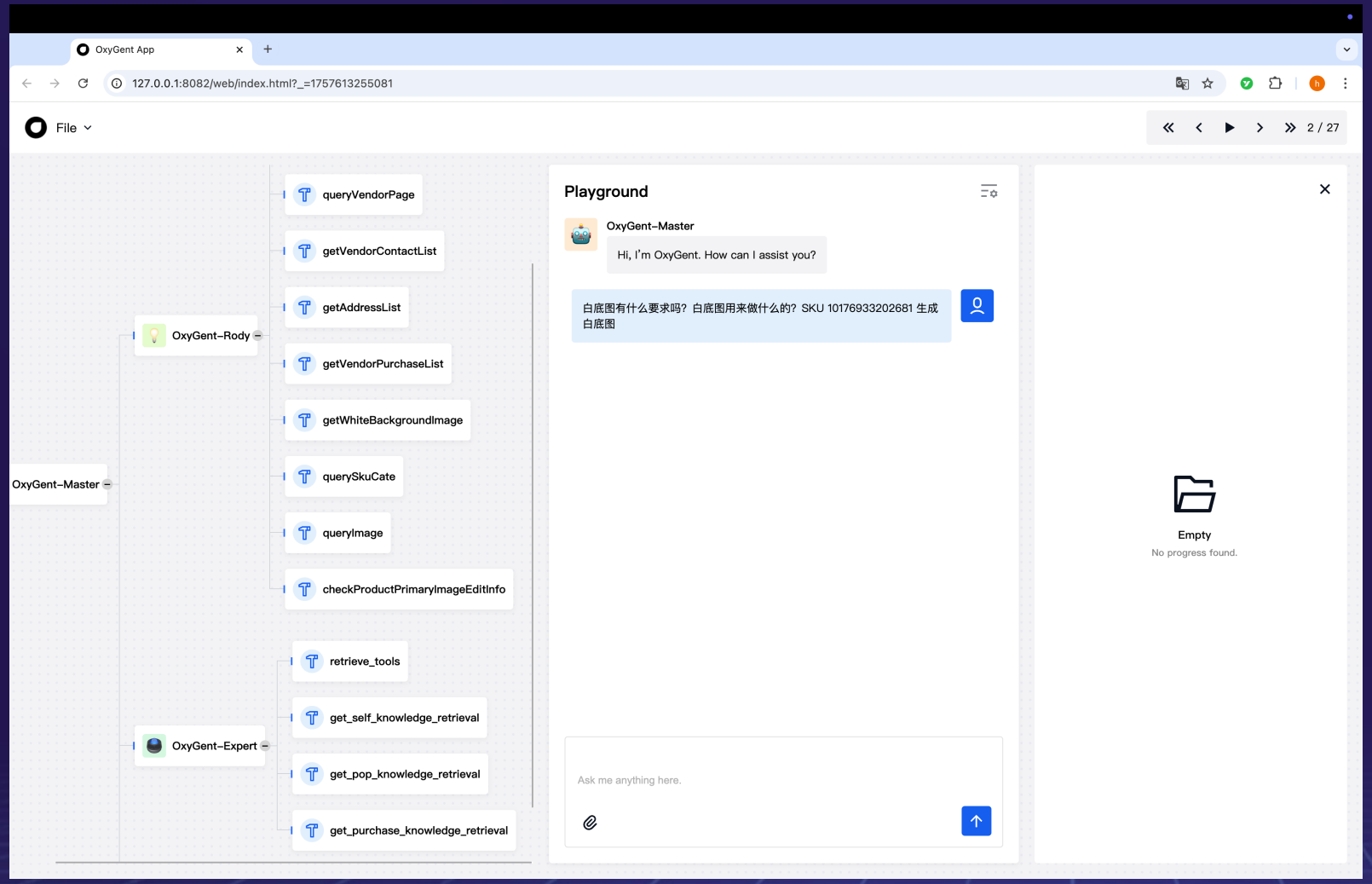
OxyGent 赋能多项业务

Agentic AICon

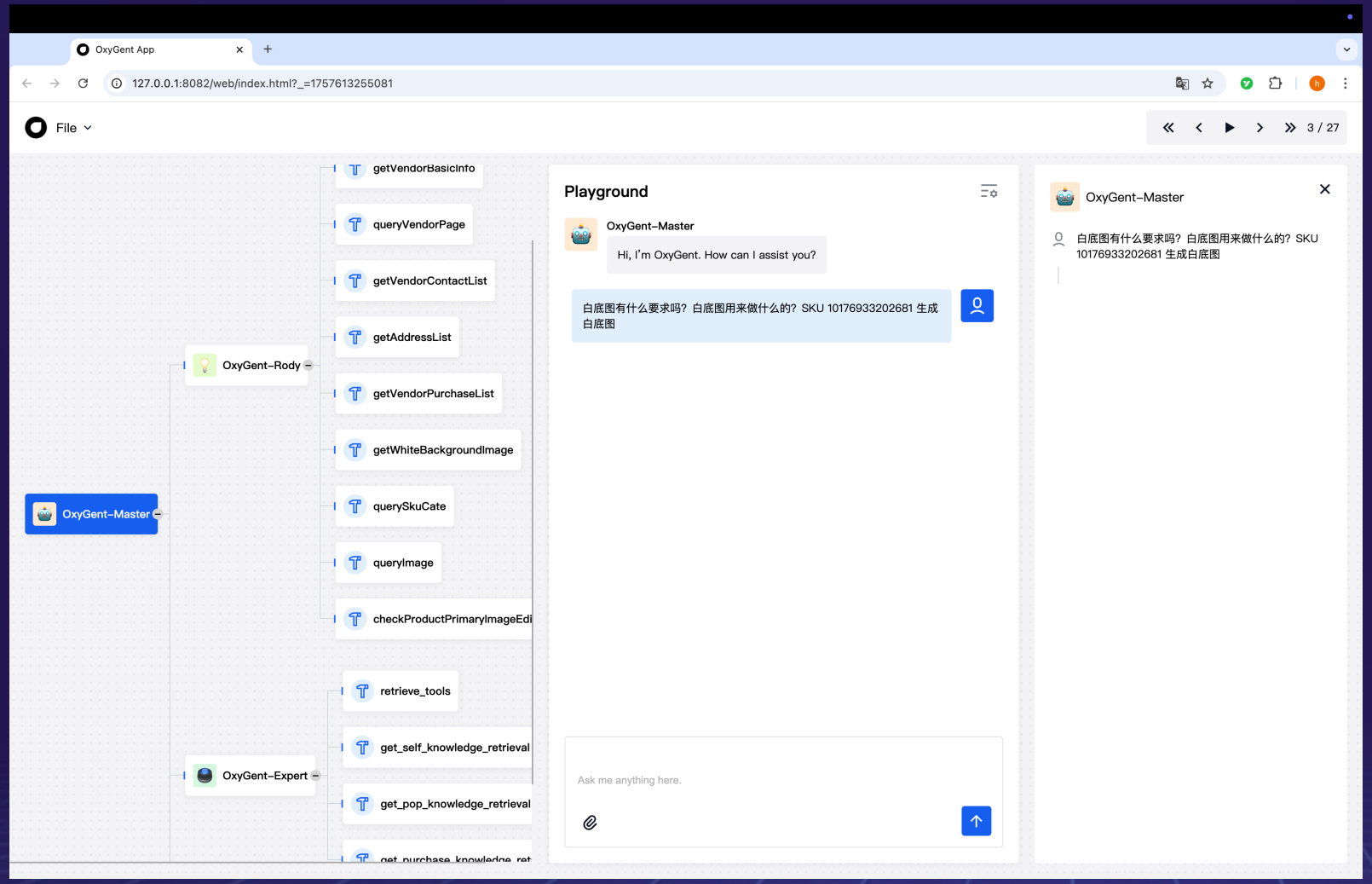
真实案例 —— 经营助手


OxyGent App
127.0.0.1:8082/web/index.html?_=1757613255081
File
1 / 27
queryVendorPage
getVendorContactList
getAddressList
OxyGent–Rody
getVendorPurchaseList
getWhiteBackgroundImage
OxyGent–Master
querySkuCate
queryImage
checkProductPrimaryImageEditInfo
retrieve_tools
get_self_knowledge_retrieval
OxyGent–Expert
get_pop_knowledge_retrieval
get_purchase_knowledge_retrieval
Playground
OxyGent–Master
Hi, I'm OxyGent. How can I assist you?
Ask me anything here.
Empty
No progress found.

真实案例 —— 经营助手


OxyGent App
127.0.0.1:8082/web/index.html?_=1757613255081
File
2 / 27
queryVendorPage
getVendorContactList
getAddressList
OxyGent-Rody
getVendorPurchaseList
getWhiteBackgroundImage
OxyGent-Master
querySkuCate
queryImage
checkProductPrimaryImageEditInfo
retrieve_tools
get_self_knowledge_retrieval
OxyGent-Expert
get_pop_knowledge_retrieval
get_purchase_knowledge_retrieval
Playground
OxyGent-Master
Hi, I'm OxyGent. How can I assist you?
白底图有什么要求吗？白底图用来做什么的？SKU 10176933202681 生成白底图
Ask me anything here.
Empty
No progress found.

真实案例 —— 经营助手



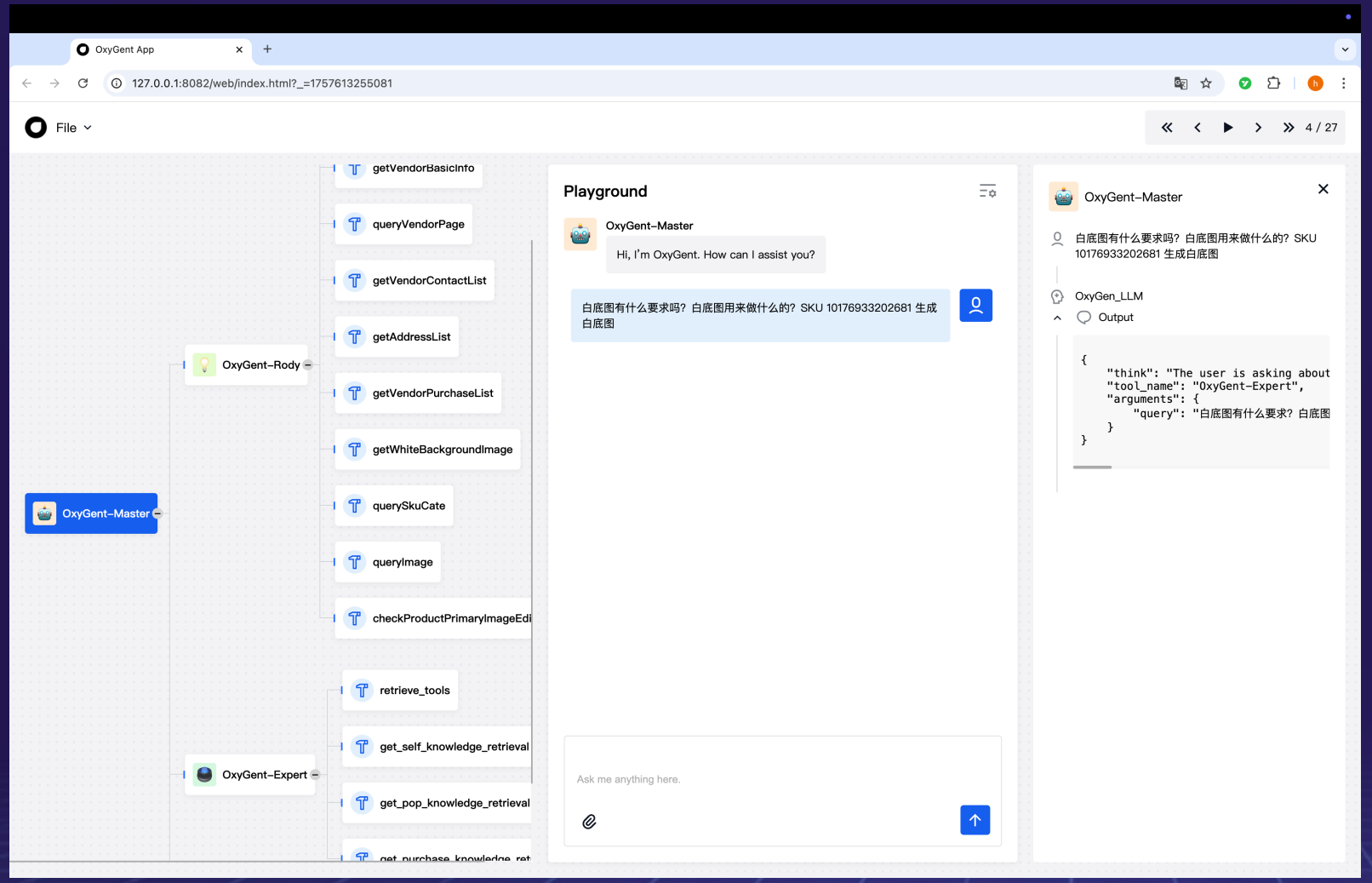
Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手

Agentic AICon

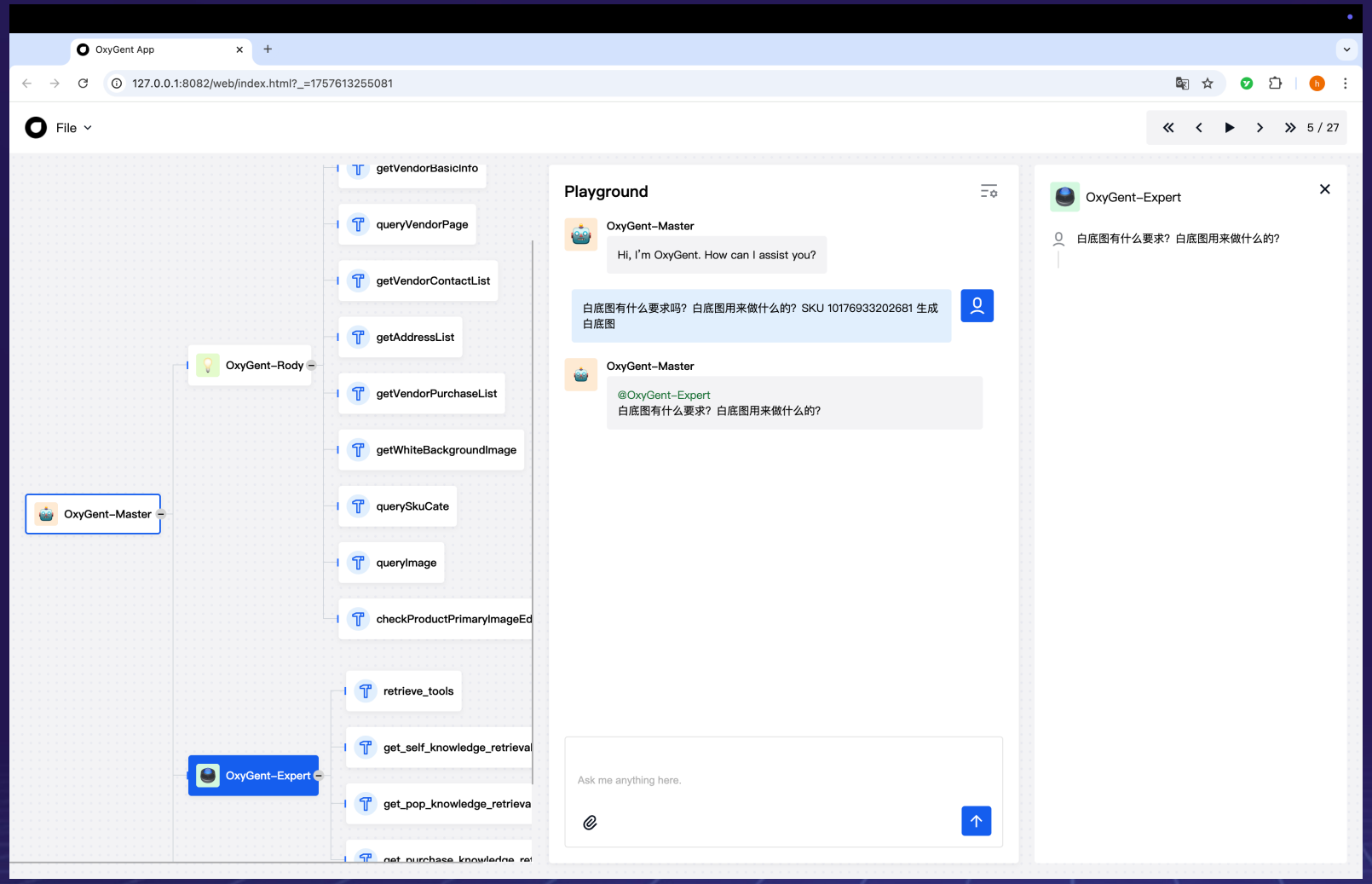


Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手



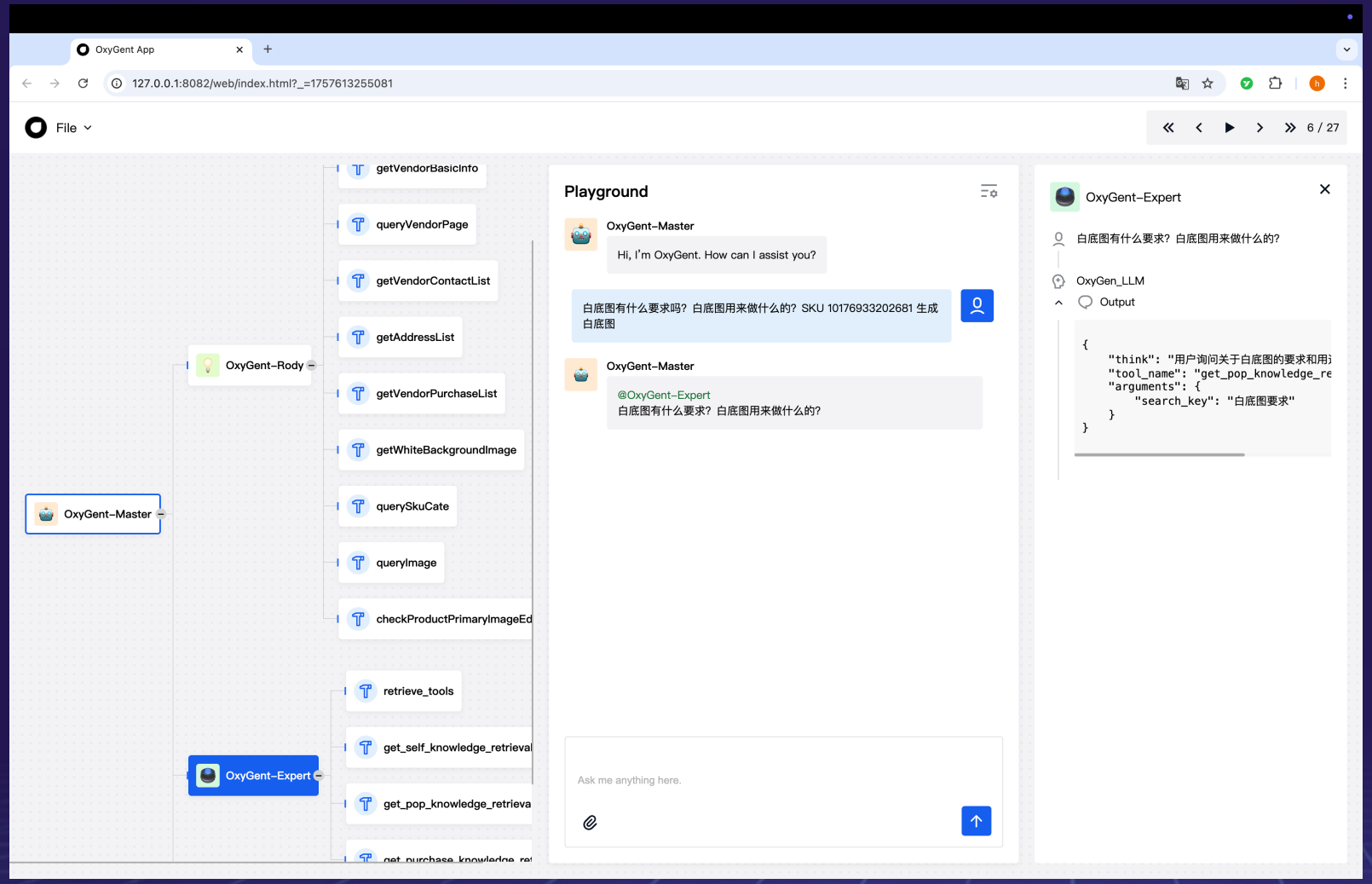
Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手

Agentic AICon



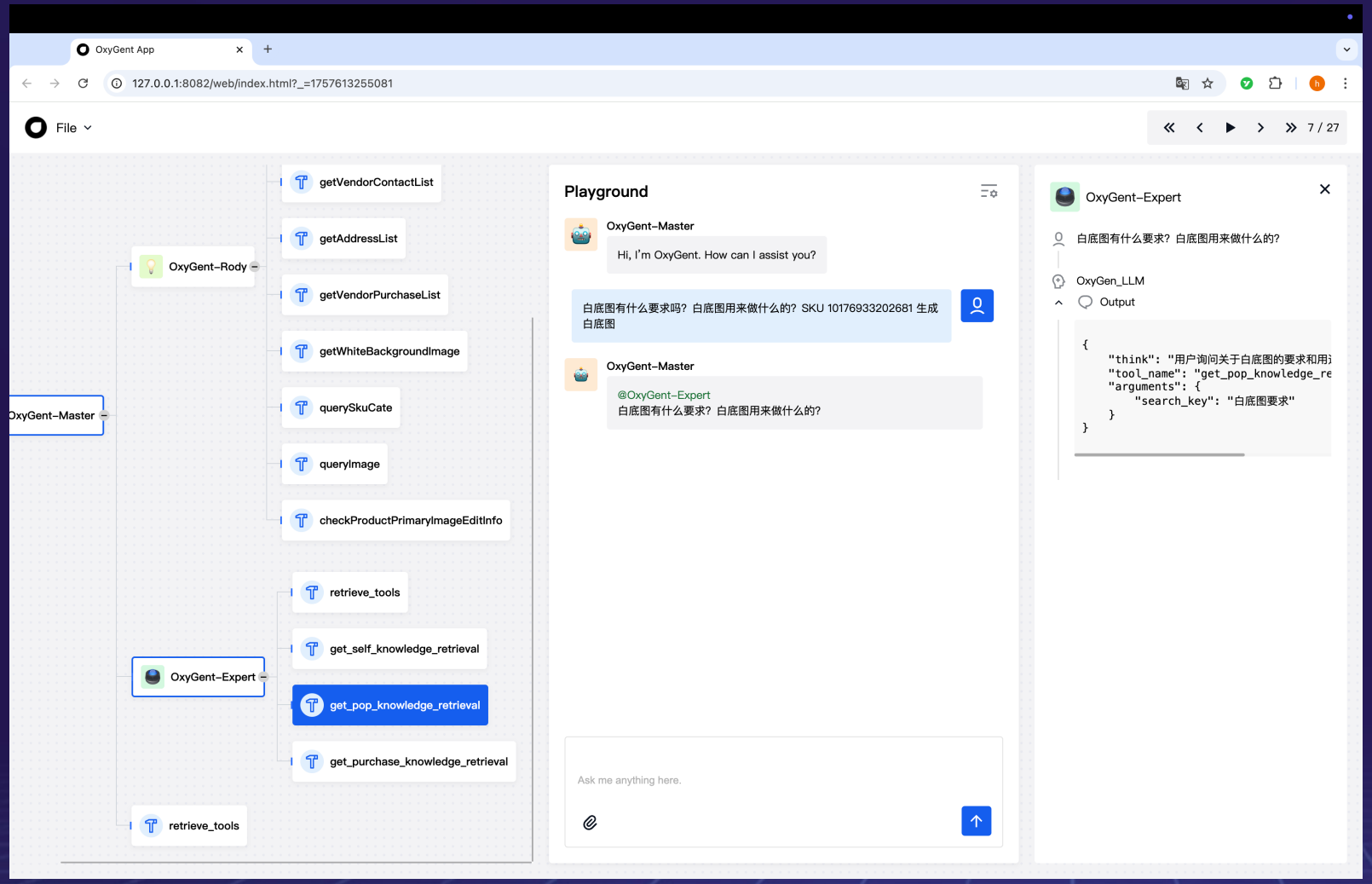
Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手

Agentic AICon

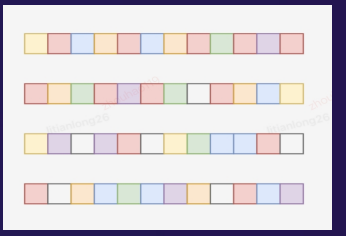


Master 规划

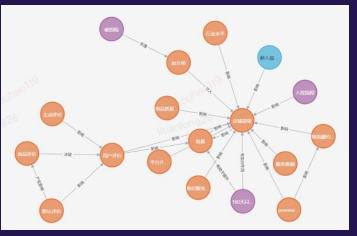
Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

多路召回



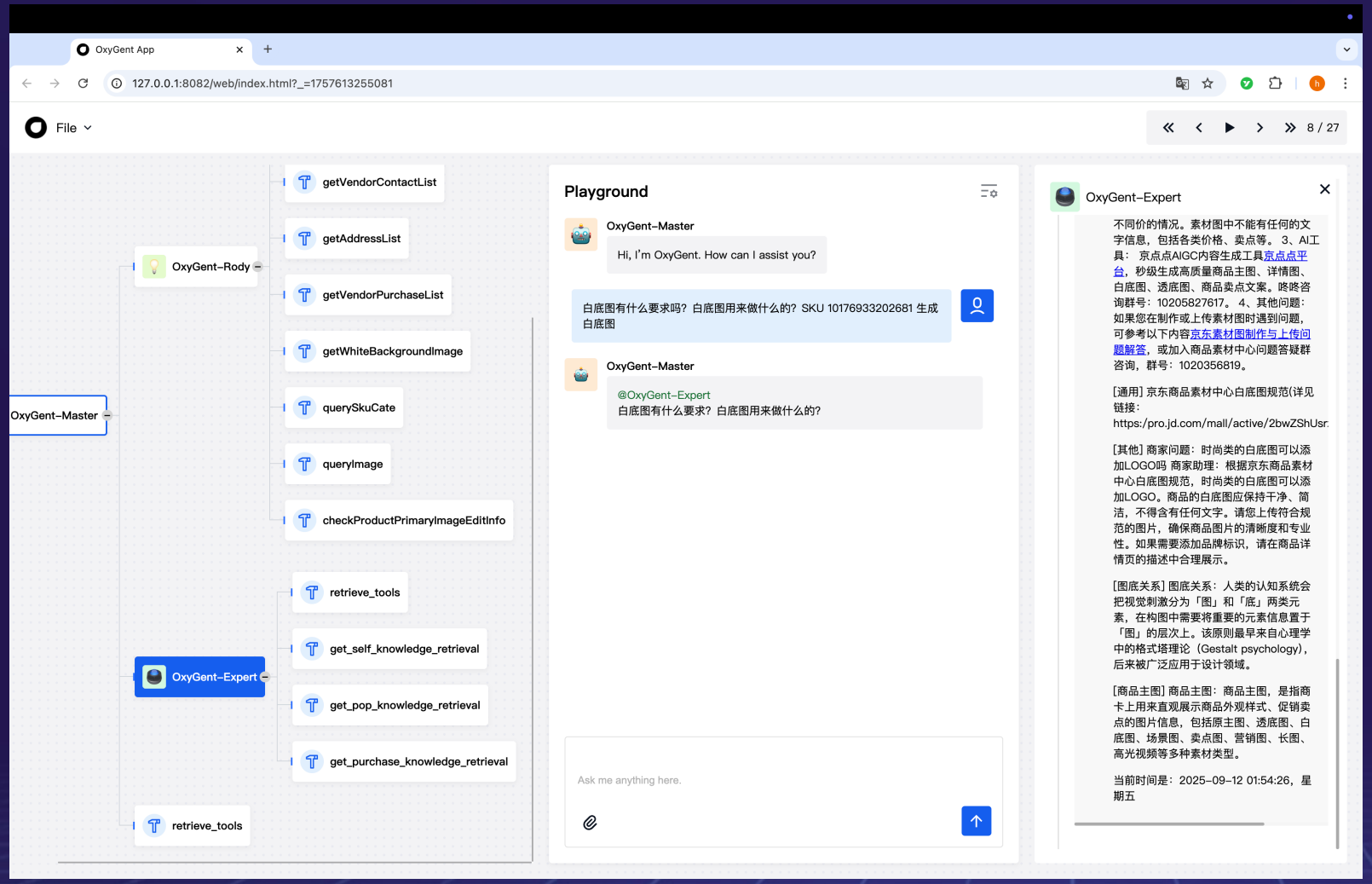
向量数据库



图数据库

真实案例 —— 经营助手

Agentic AICon



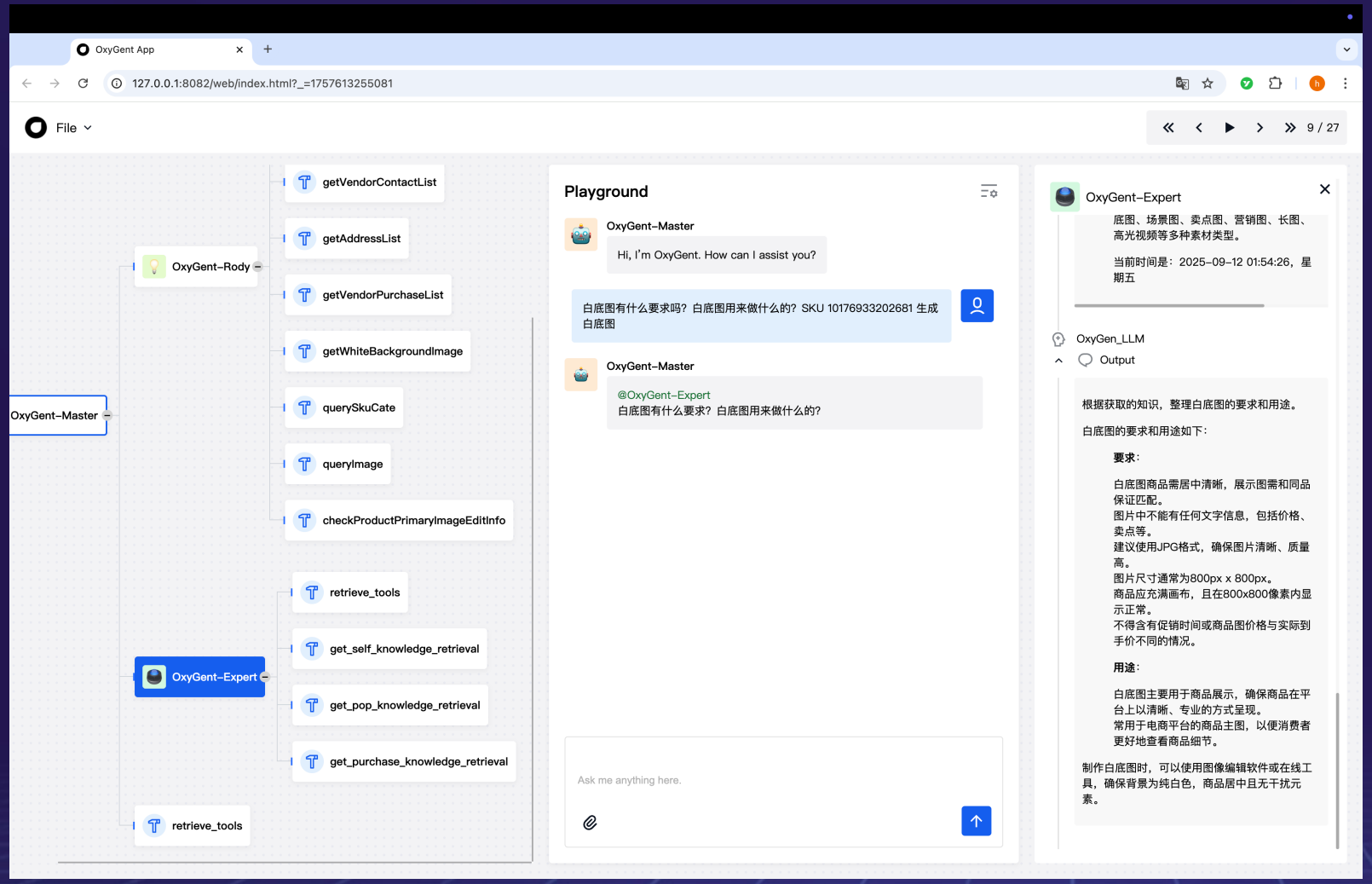
Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手

Agentic AICon



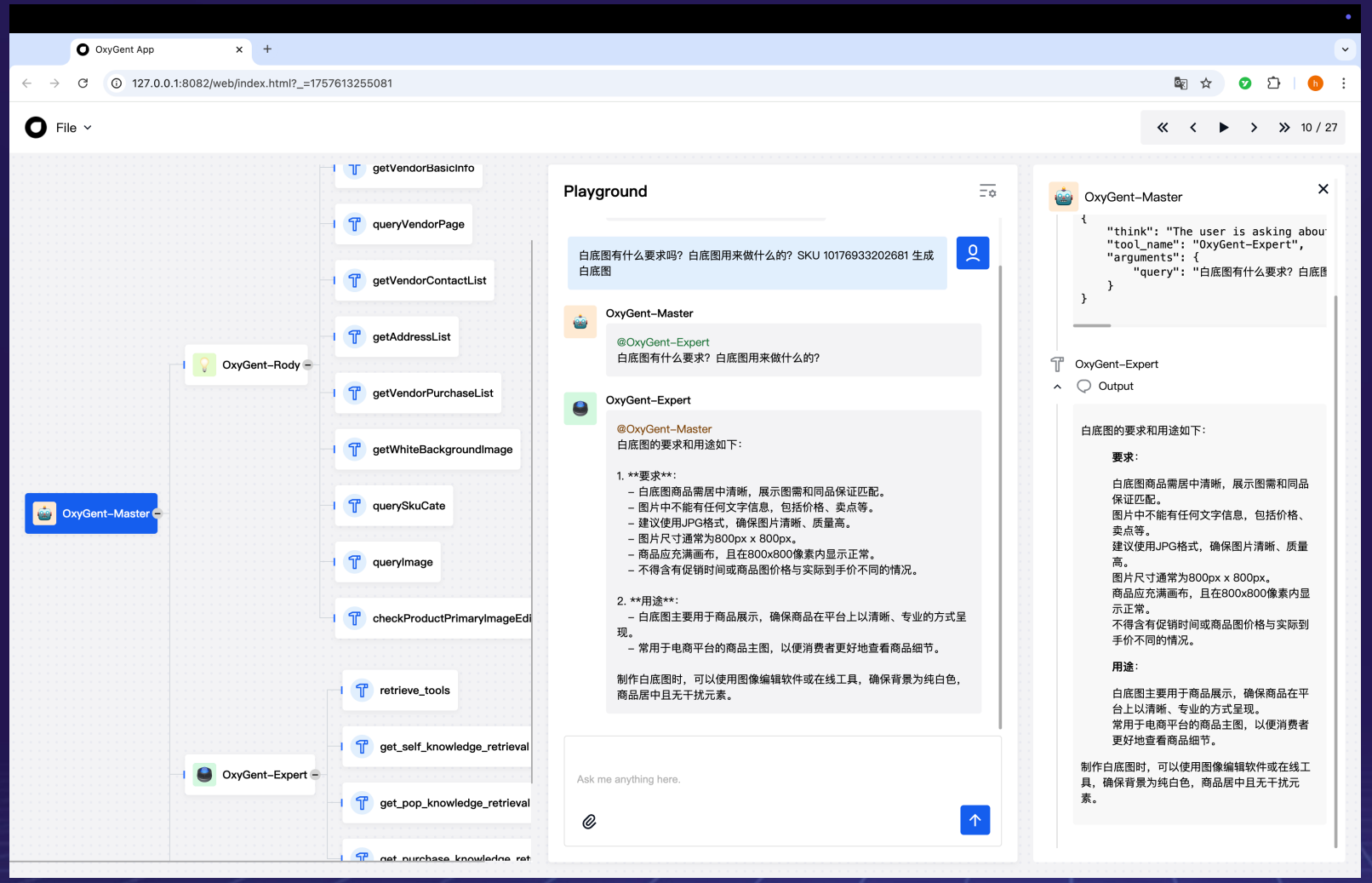
Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手

Agentic AICon



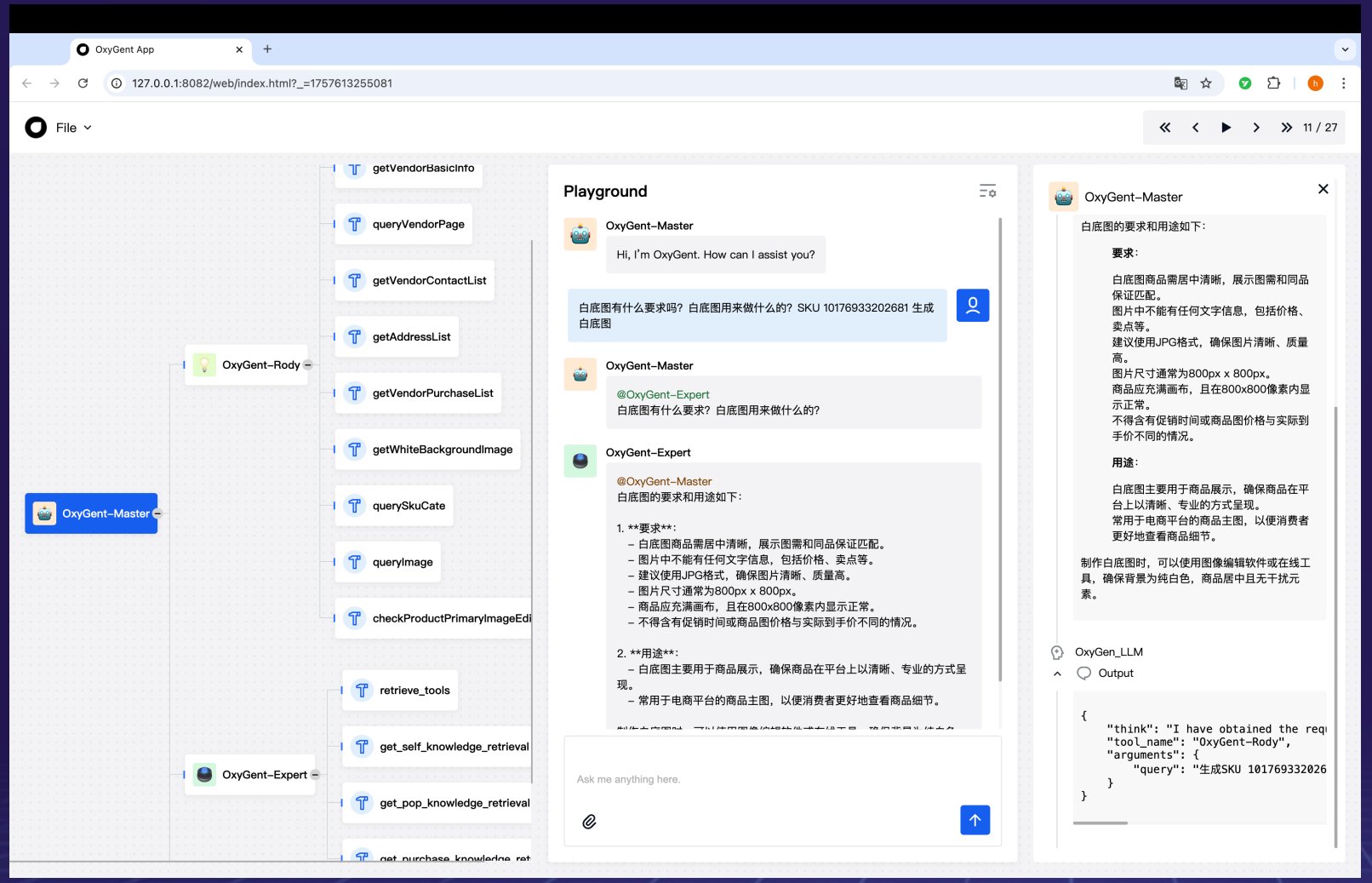
Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手

Agentic AICon



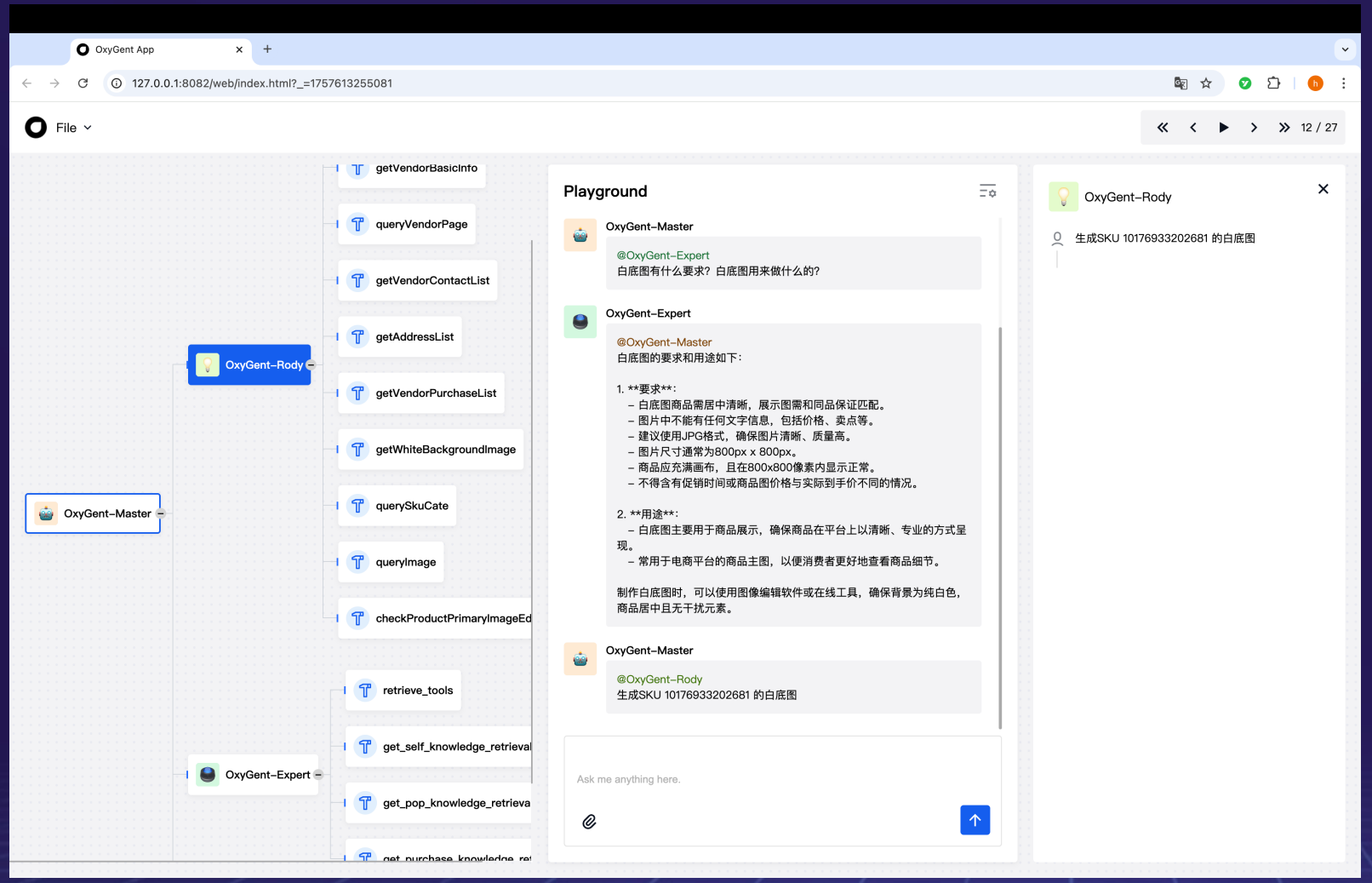
Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手

Agentic AICon



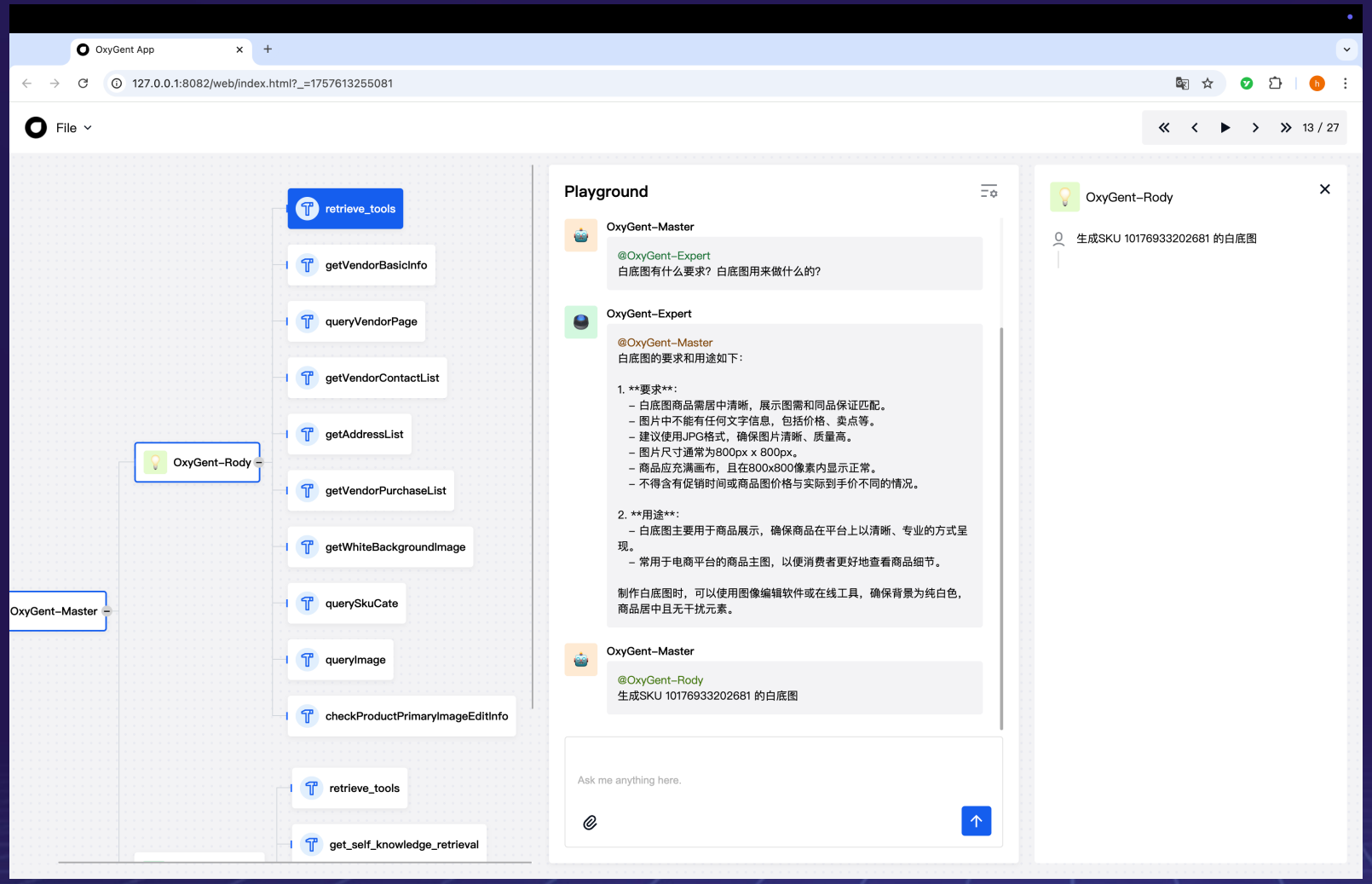
Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手

Agentic AICon



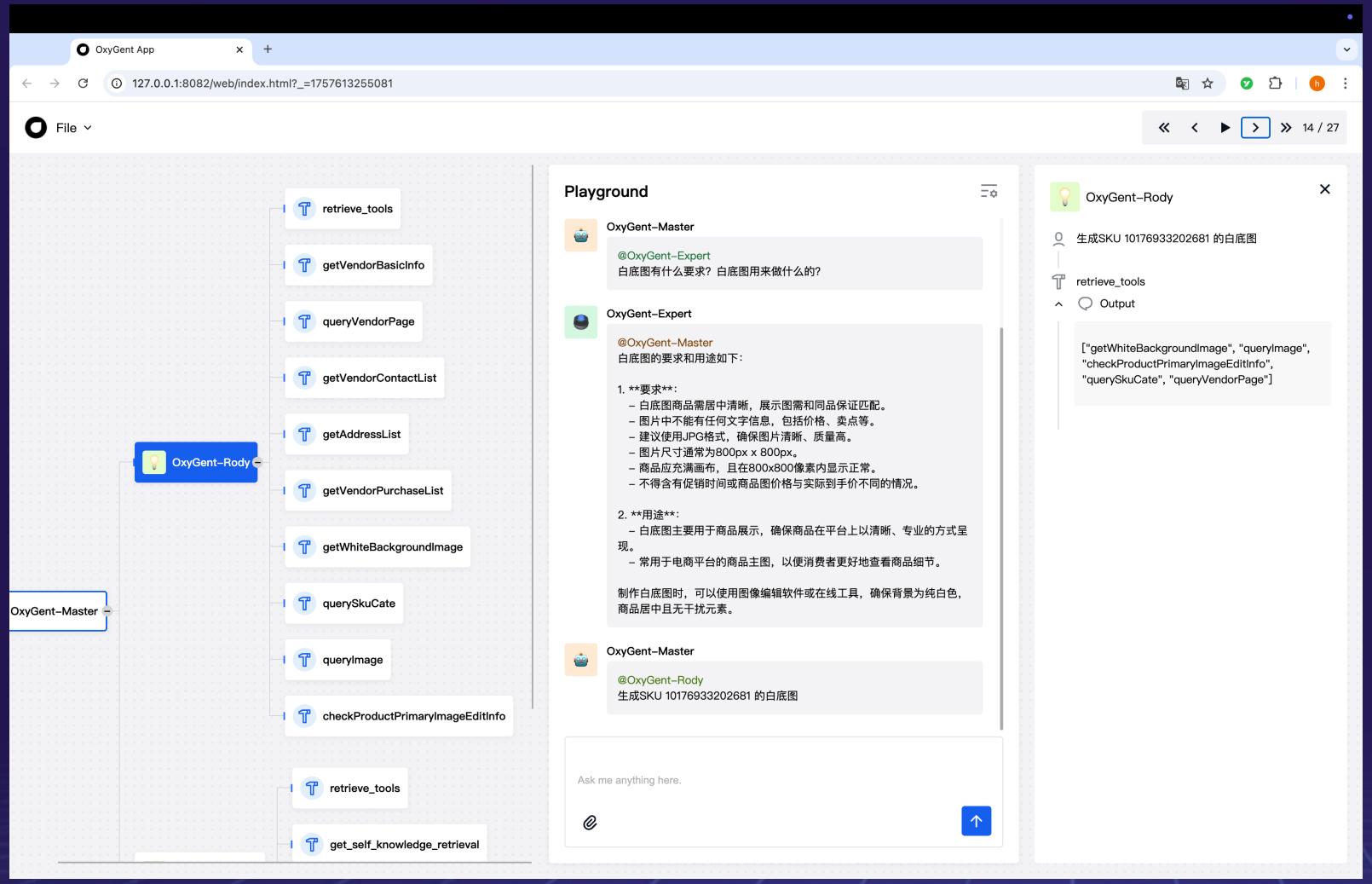
Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手

Agentic AICon



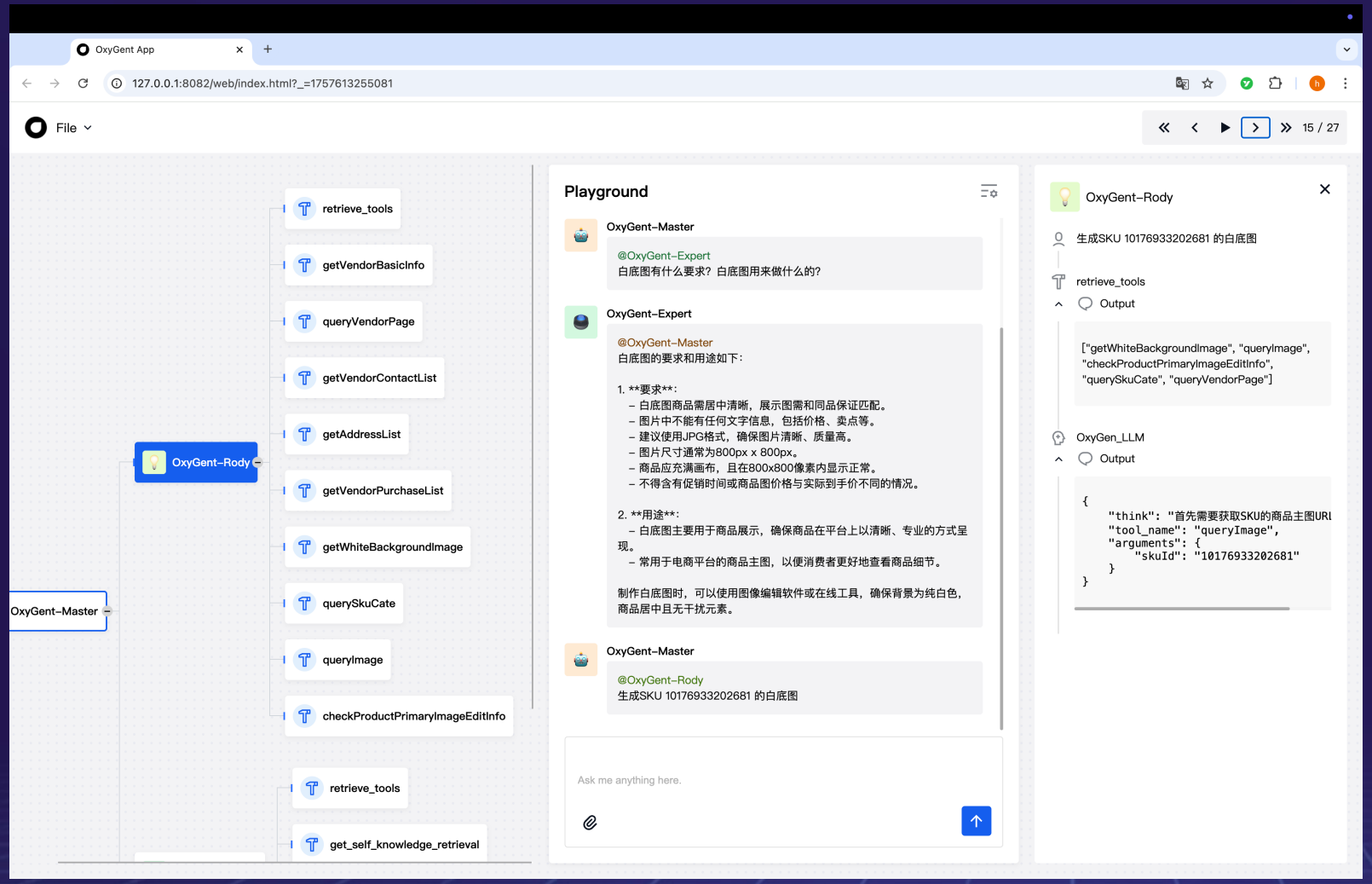
Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手

Agentic AICon

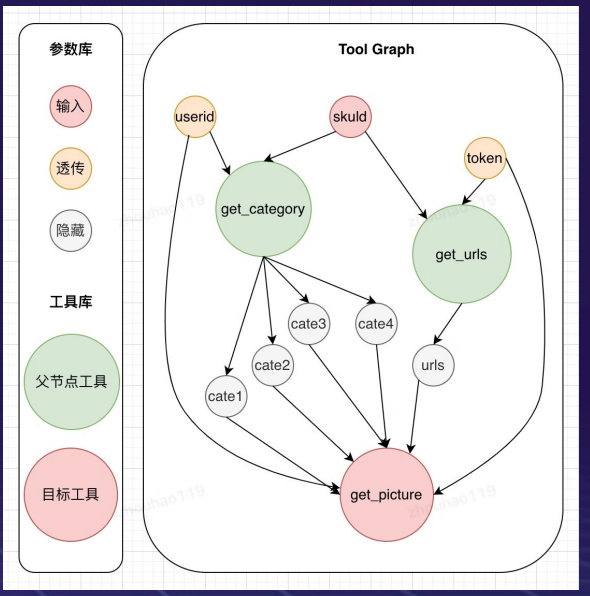


Master 规划

Step 1: 查找【白底图】知识点

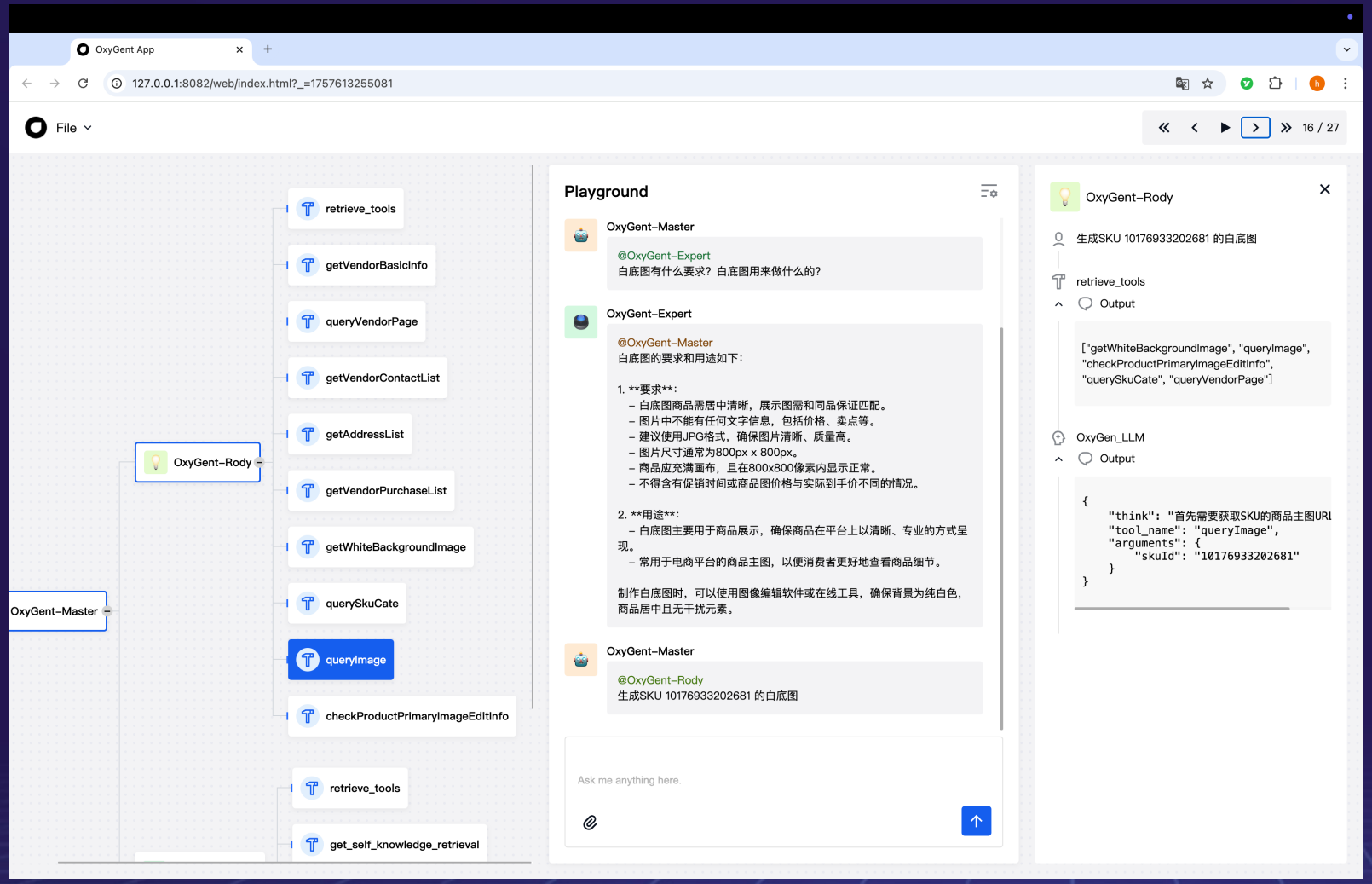
Step 2: 调用【白底图生成】工具

逆向规划



真实案例 —— 经营助手

Agentic AICon



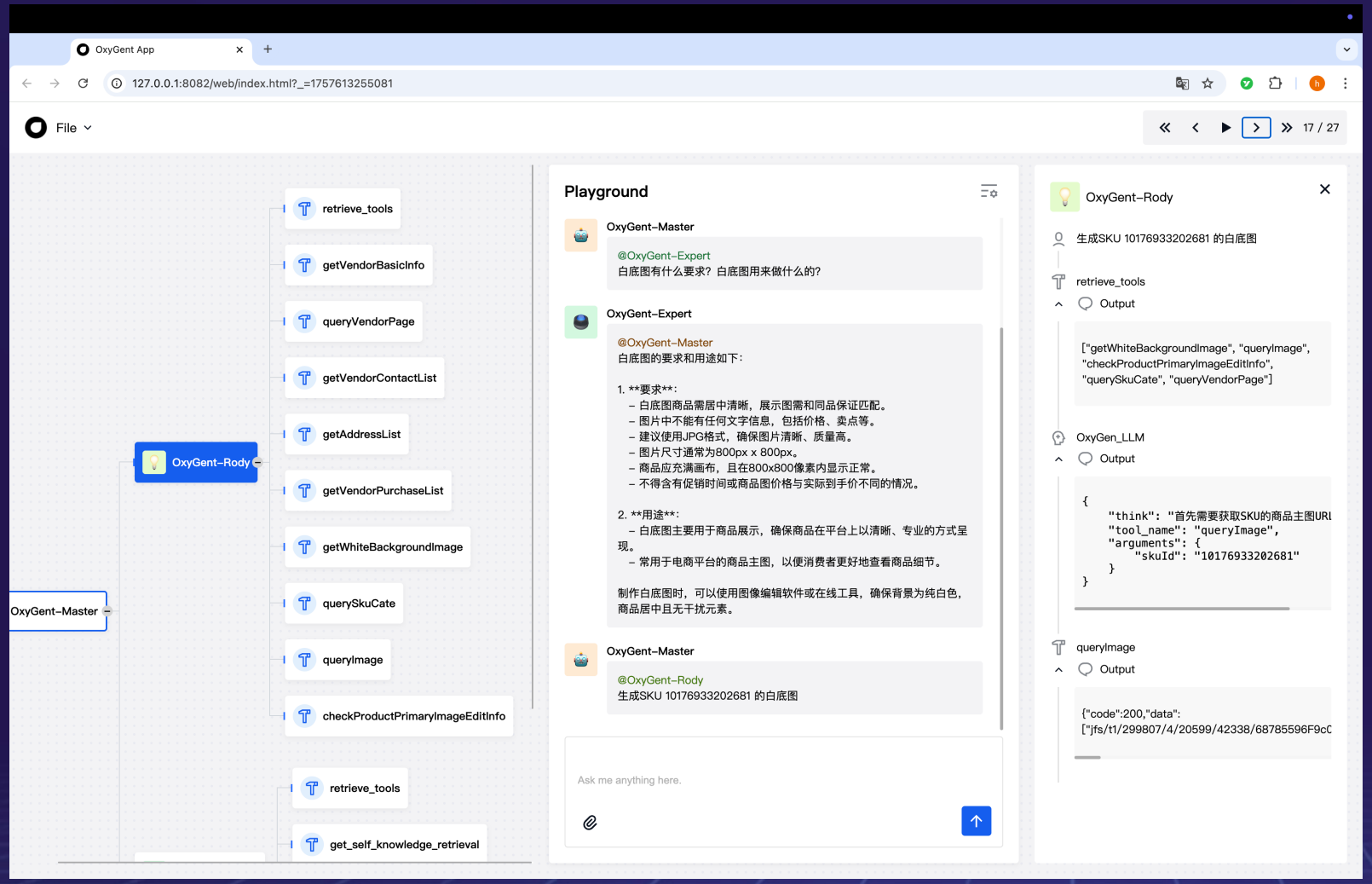
Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手

Agentic AICon



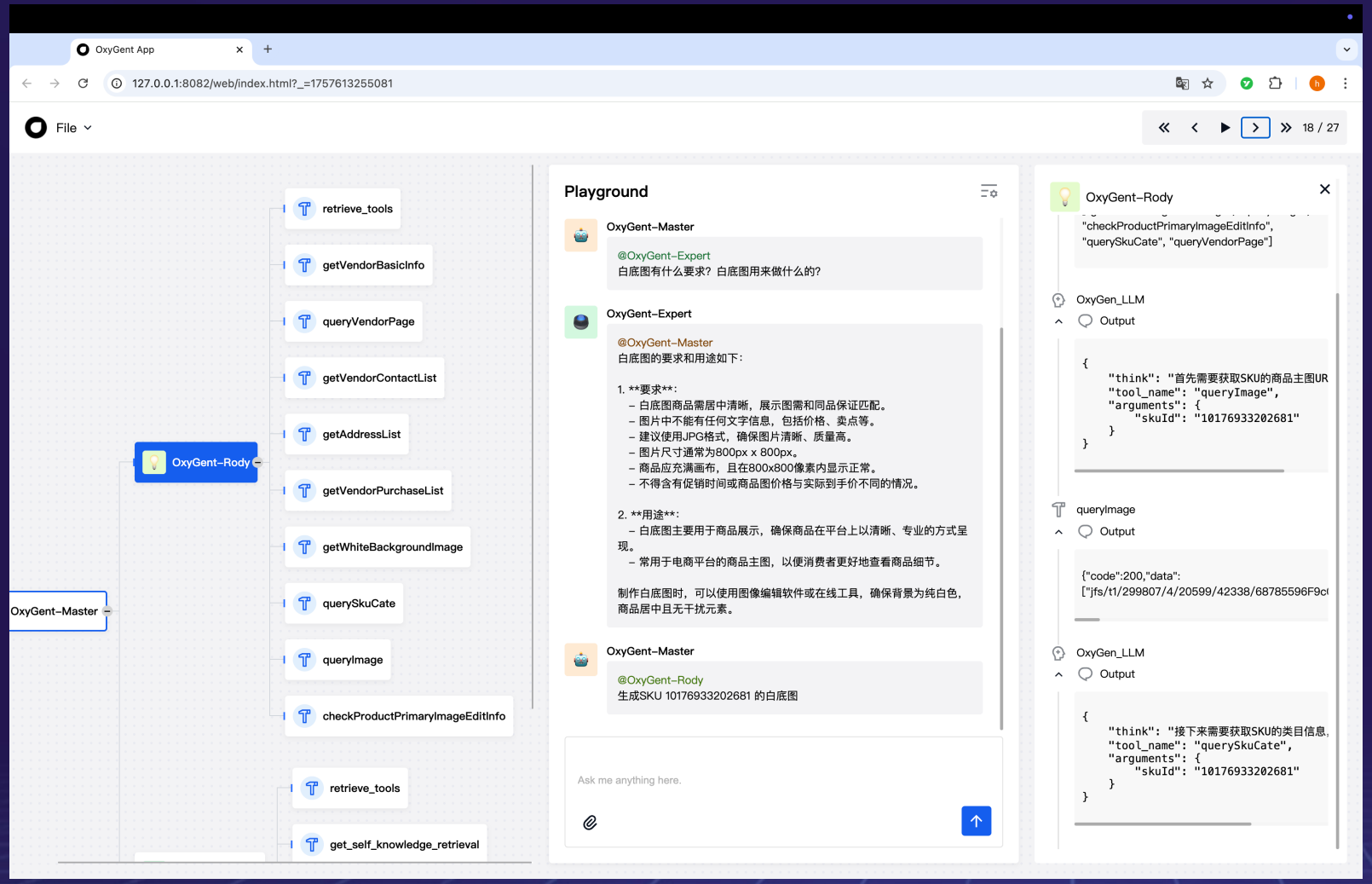
Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手

Agentic AICon



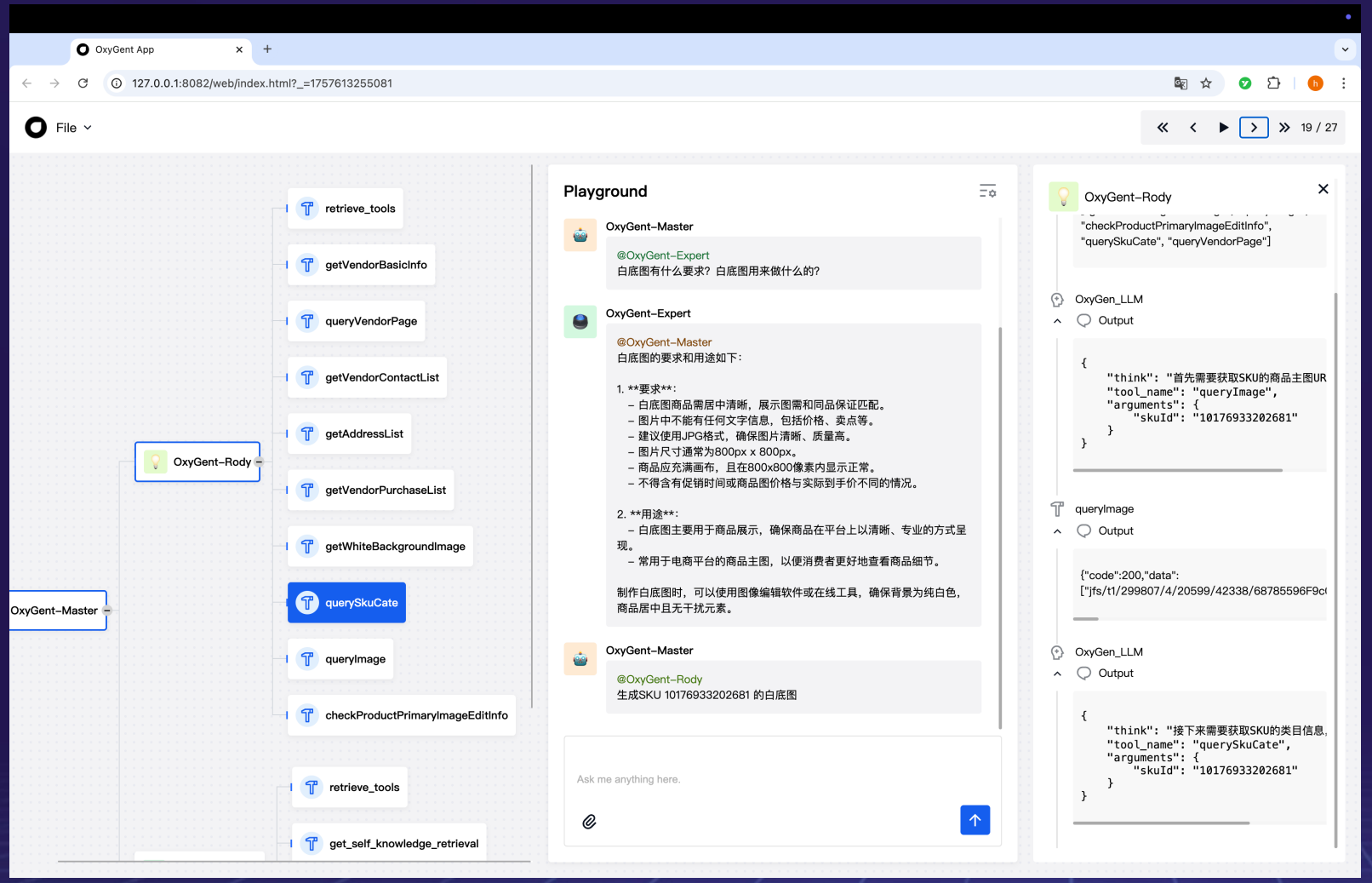
Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手

Agentic AICon



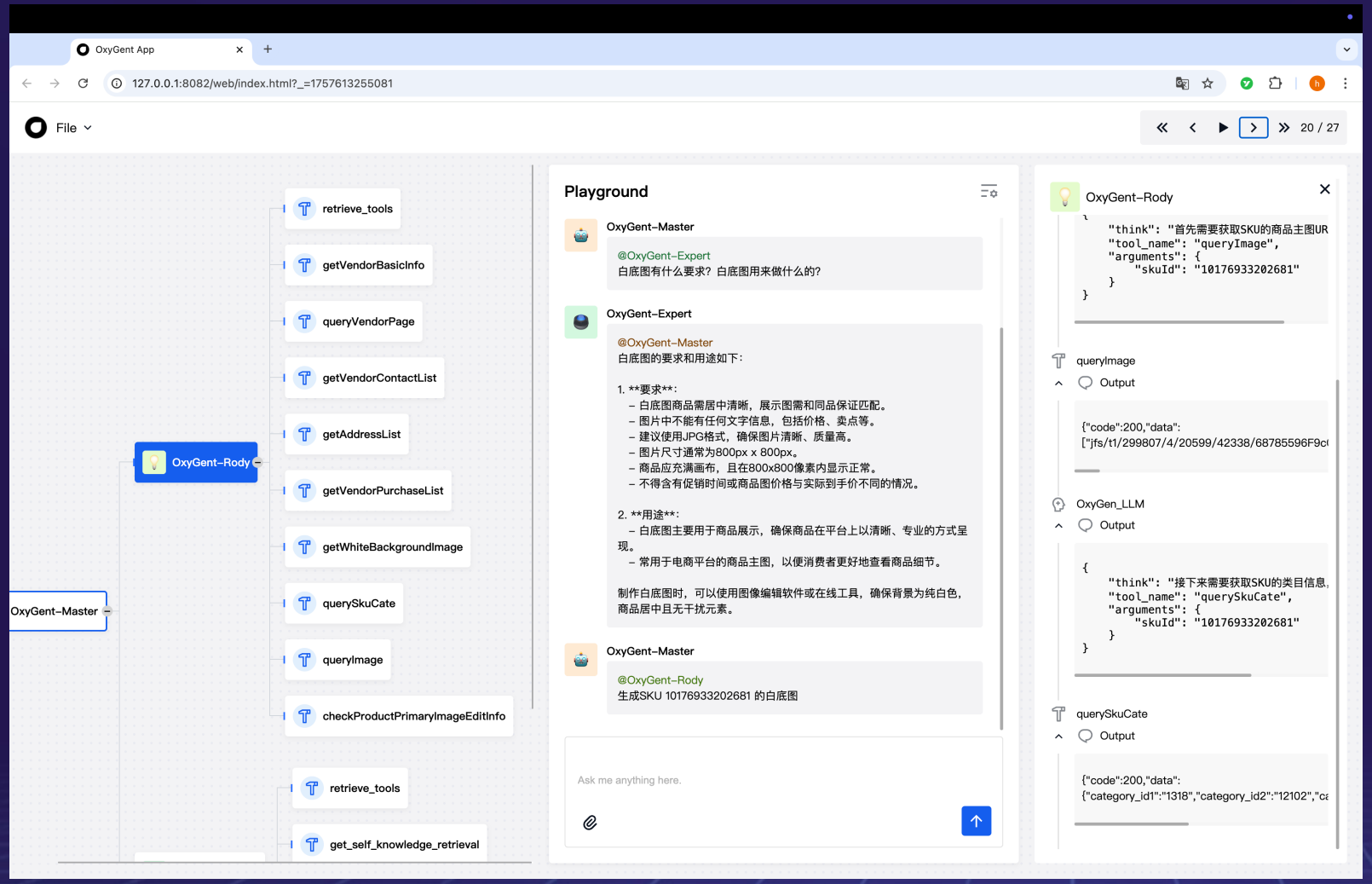
Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手

Agentic AICon



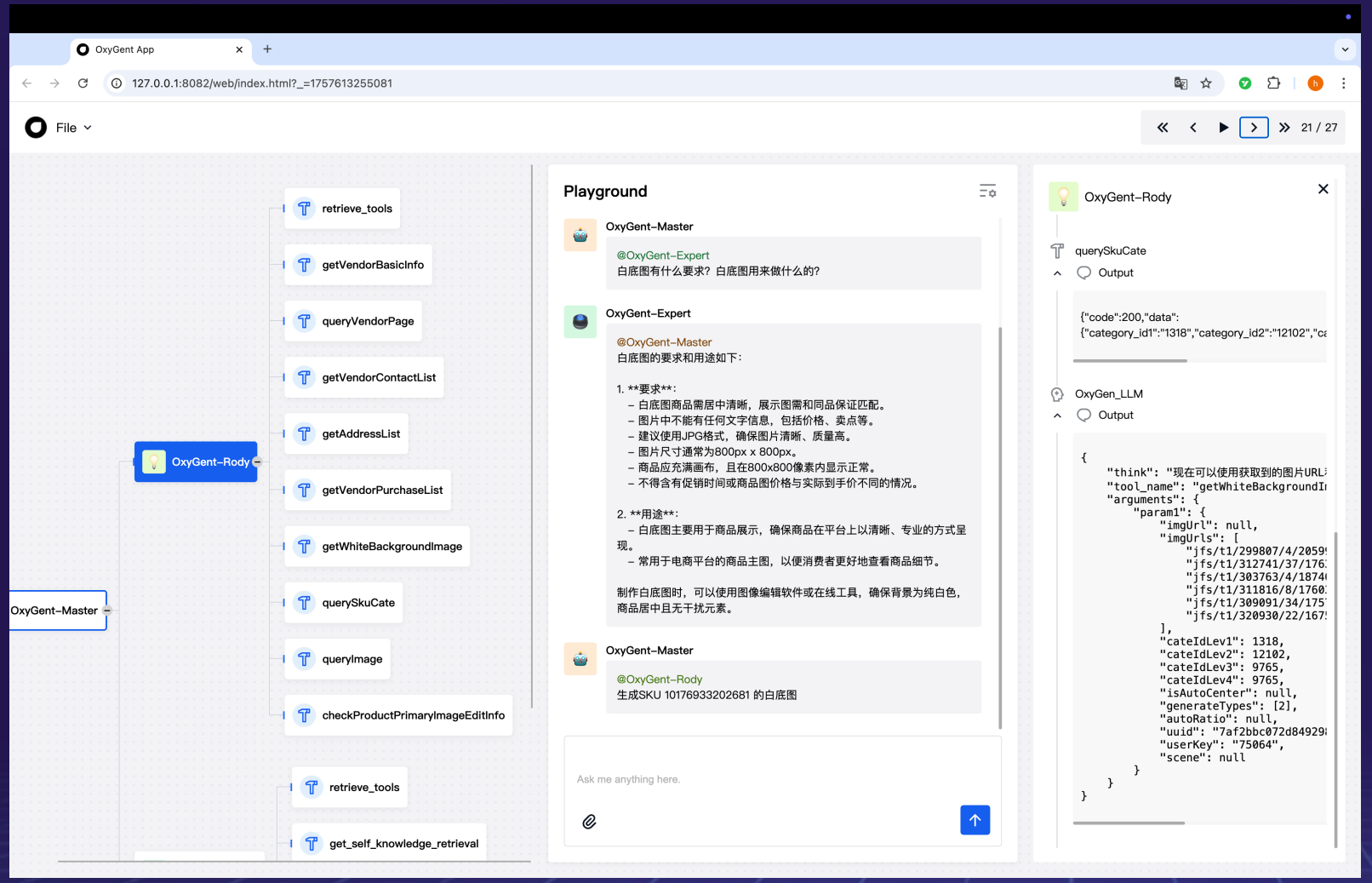
Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手

Agentic AICon



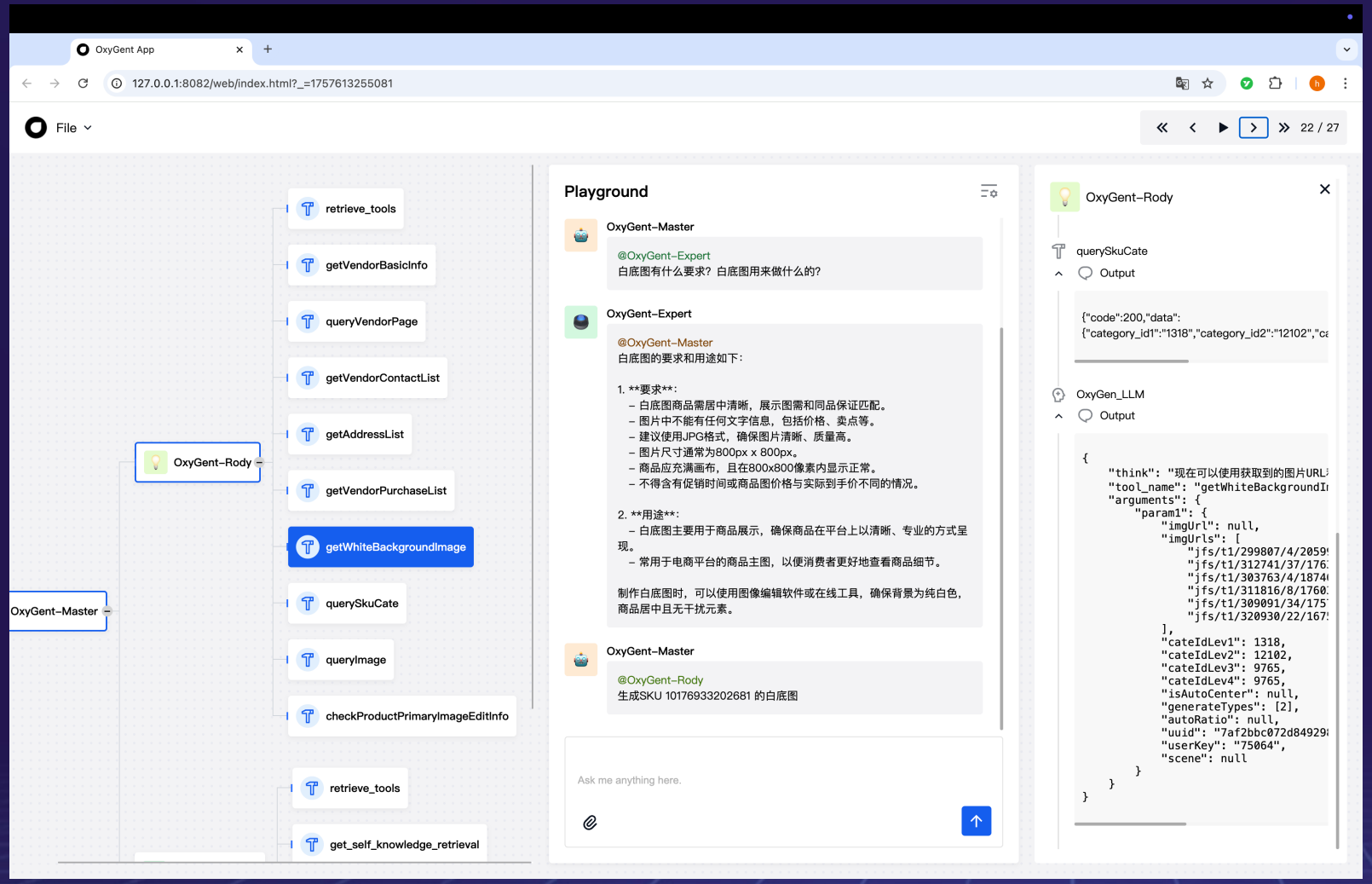
Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手

Agentic AICon



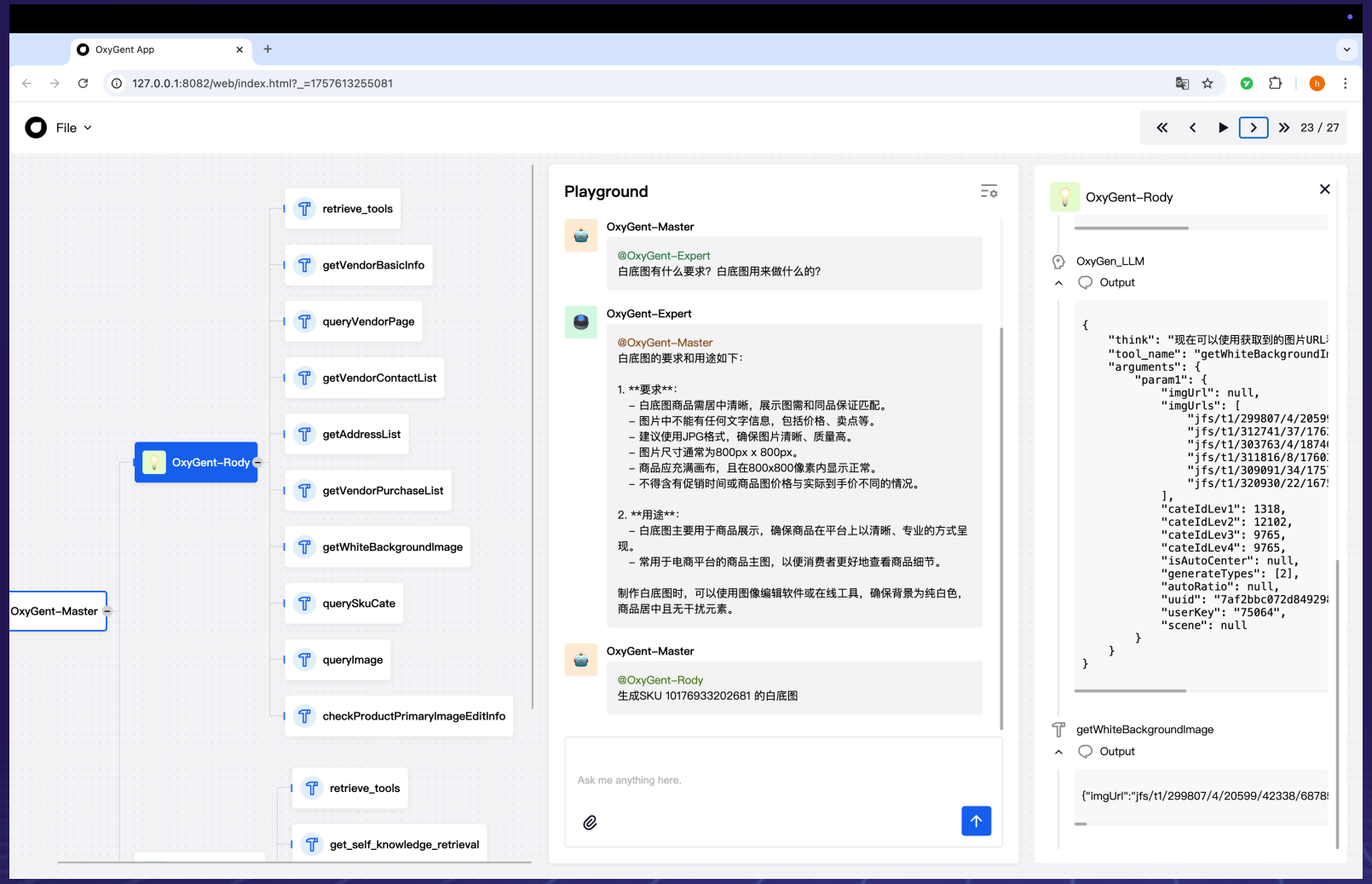
Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手

Agentic AICon



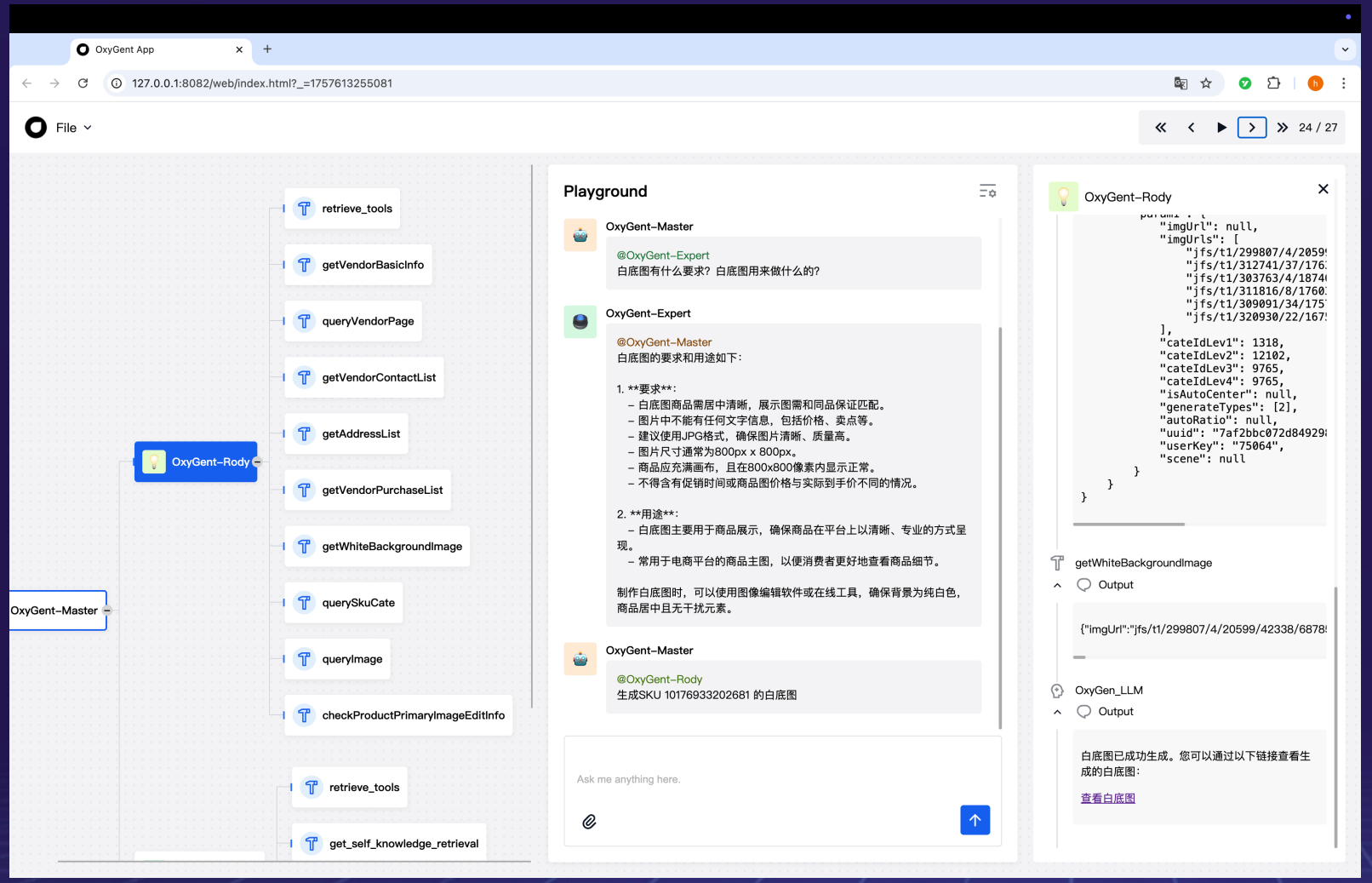
Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手

Agentic AICon



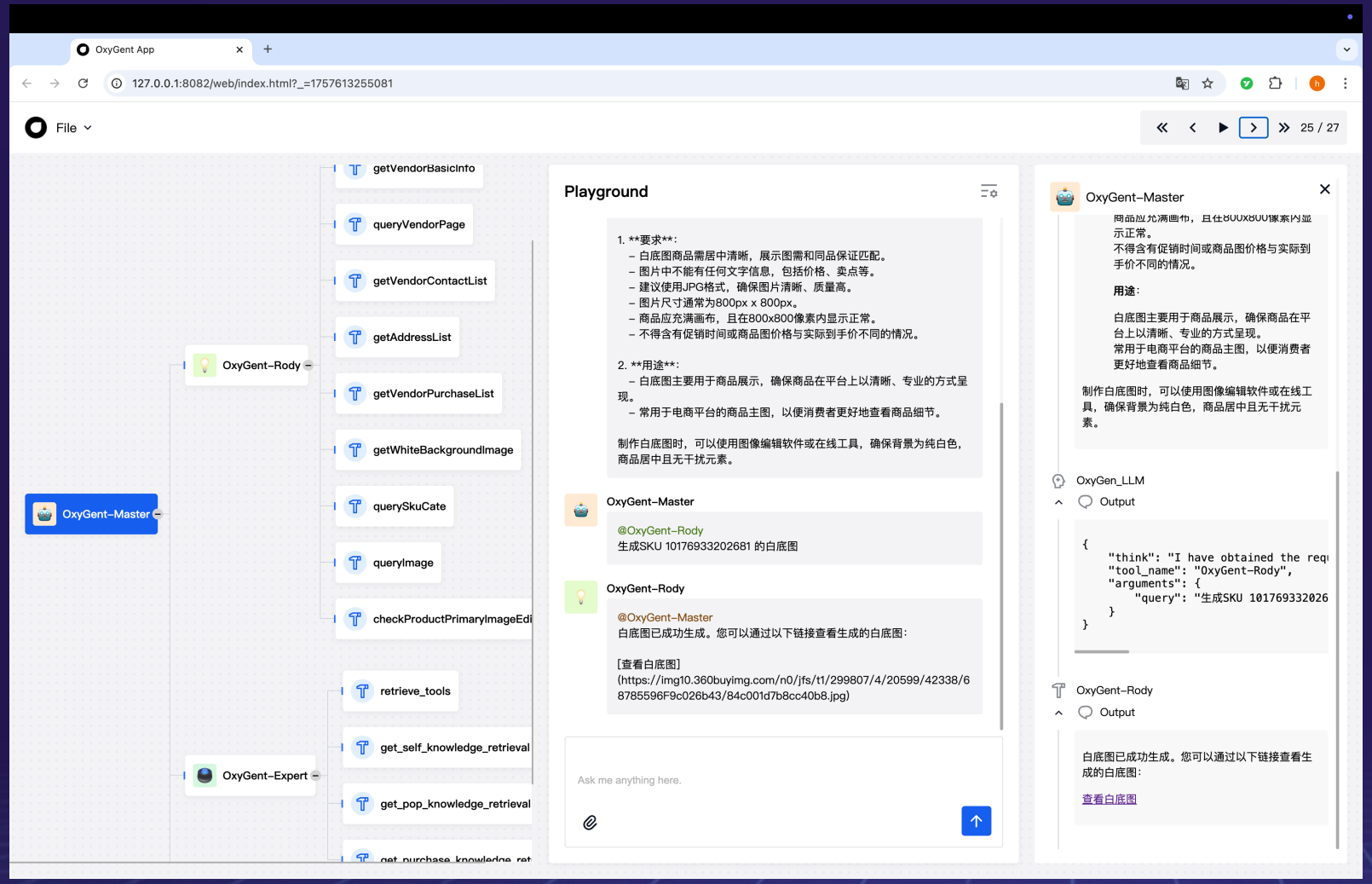
Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手

Agentic AICon



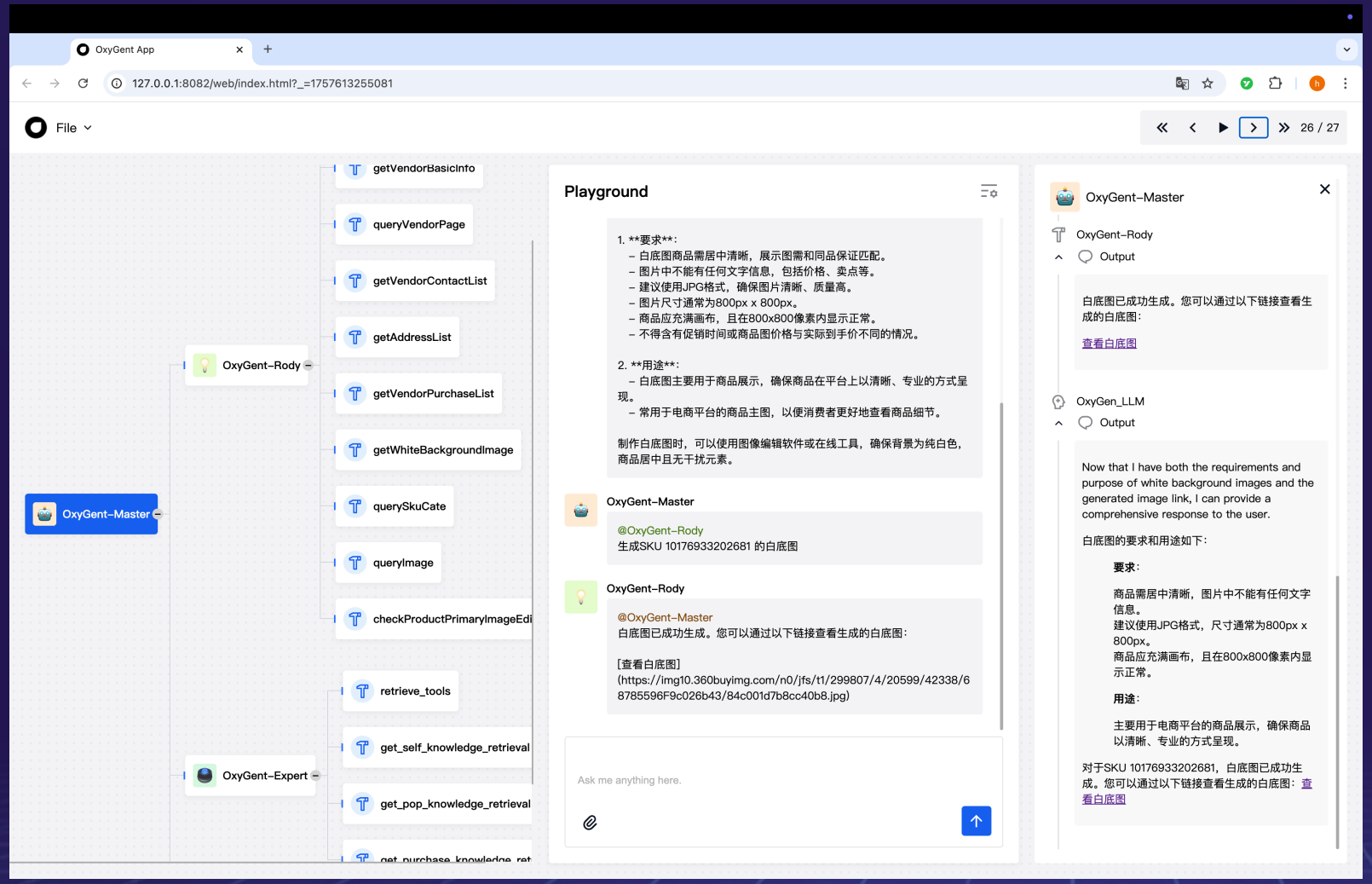
Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手

Agentic AICon

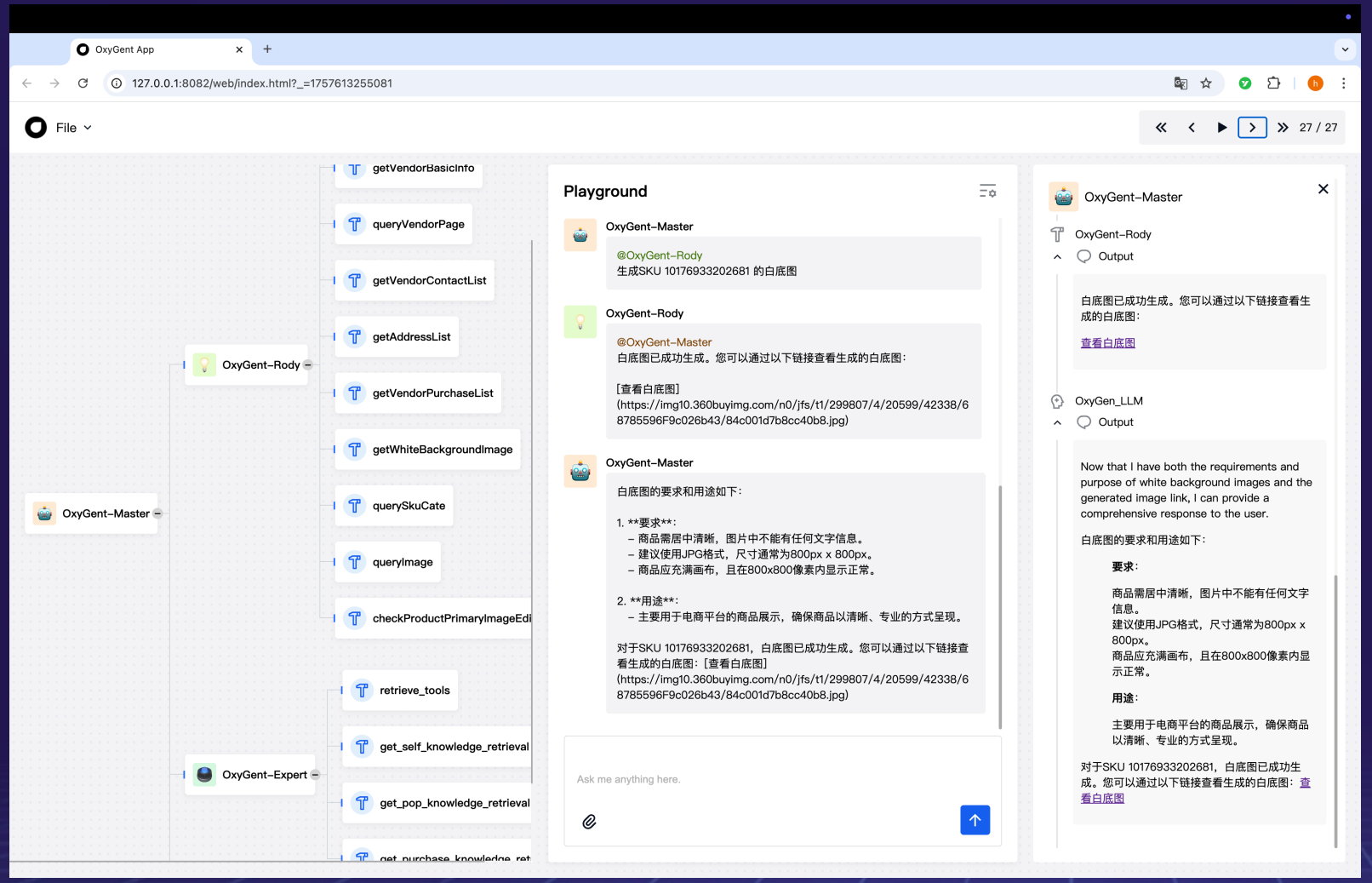


Master 规划

Step 1: 查找【白底图】知识点

Step 2: 调用【白底图生成】工具

真实案例 —— 经营助手


OxyGent App
127.0.0.1:8082/web/index.html?_=1757613255081
File
27 / 27
getVendorBasicInfo
queryVendorPage
getVendorContactList
getAddressList
OxyGent–Rody
getVendorPurchaseList
getWhiteBackgroundImage
OxyGent–Master
querySkuCate
queryImage
checkProductPrimaryImageEdi
retrieve_tools
get_self_knowledge_retrieval
OxyGent–Expert
get_pop_knowledge_retrieval
Playground
OxyGent–Master
@OxyGent–Rody
生成SKU 10176933202681 的白底图
OxyGent–Rody
@OxyGent–Master
白底图已成功生成。您可以通过以下链接查看生成的白底图：
[查看白底图]
(https://img10.360buyimg.com/n0/jfs/t1/299807/4/20599/42338/68785596F9c026b43/84c001d7b8cc40b8.jpg)
OxyGent–Master
白底图的要求和用途如下：
1. **要求**：
– 商品需居中清晰，图片中不能有任何文字信息。
– 建议使用JPG格式，尺寸通常为800px x 800px。
– 商品应充满画布，且在800x800像素内显示正常。
2. **用途**：
– 主要用于电商平台的商品展示，确保商品以清晰、专业的方式呈现。
对于SKU 10176933202681，白底图已成功生成。您可以通过以下链接查看生成的白底图：[查看白底图]
(https://img10.360buyimg.com/n0/jfs/t1/299807/4/20599/42338/68785596F9c026b43/84c001d7b8cc40b8.jpg)
Ask me anything here.
OxyGent–Master
OxyGent–Rody
Output
白底图已成功生成。您可以通过以下链接查看生成的白底图：
查看白底图
OxyGen_LLM
Output
Now that I have both the requirements and purpose of white background images and the generated image link, I can provide a comprehensive response to the user.
白底图的要求和用途如下：
要求：
商品需居中清晰，图片中不能有任何文字信息。
建议使用JPG格式，尺寸通常为800px x 800px。
商品应充满画布，且在800x800像素内显示正常。
用途：
主要用于电商平台的商品展示，确保商品以清晰、专业的方式呈现。
对于SKU 10176933202681，白底图已成功生成。您可以通过以下链接查看生成的白底图：查看白底图

进阶 - L4自主经营

Agentic AICon

动作空间：　　提报营销活动、改价格、投广告、改素材等

约束：　　设置底线价、广告日预算等

执行权限：　　手动执行、自动执行

回滚机制：　　执行后监控指标，如果与预期差异较大，直接触发动作回滚

用户设置目标约束“帮我把这批临期品清仓，亏损不能超过15%”

Agent自主经营

Take away

Agentic AICon

01

按上下文拆

多智能体的本质，是给 LLM 做「上下文减法」。拆 Agent 按上下文拆，不是按流程拆。

02

拆得开，更要接得住

状态必须隔离、跨进程要强类型契约；路由分错要能拒绝反问，而不是硬执行。

03

看得清，才敢放手

长链路必须可观测；落地先划自主边界，想清楚业务目前该落在 L 几，按 L0–L4 逐级上移。

回到上下文 —— 编排、调度、状态、可观测，都是为了让 LLM 在有限上下文里把事做对。

AgenticAICon

智能体应用与架构工程大会

感谢聆听

关注公众号

更多会议详情